

Machine Learning for Biology and Health

CSCI 1851
Spring 2026

Ritambhara Singh

March 12, 2026
Thursday



Muddy points: Batching

```
#Load all data

transform = transforms.ToTensor()

train_data = PneumoniaMNIST(split="train", transform=transform, download=True)
test_data  = PneumoniaMNIST(split="test",  transform=transform, download=True)

train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
test_loader  = DataLoader(test_data,  batch_size=64)
```

```
N_EPOCHS = 10

for epoch in range(1, N_EPOCHS + 1):
    model.train()
    total_loss = 0

    for images, labels in train_loader:
        labels = labels.float().squeeze()      # shape (batch,), values 0.0 or 1.0

        probs = model(images).squeeze()        # predicted probabilities
        loss  = criterion(probs, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

Today's goal – Learn about activations and Convolutional Neural Networks (CNNs)

- (1) Adding layers to FCNNs
- (2) Activation functions for non-linearity
- (3) Class activity: Activation functions in action
- (4) Convolution and CNNs
- (5) Learning CNNs

Adding layers to FCNNs

Our problem:

Input: $\mathbb{X}$

Pixel Grid

$x^{(1)} =$



128x128 pixels



Function: f



$f(\mathbb{X}) \rightarrow \mathbb{Y}$

Target: $\mathbb{Y}$

Has Pneumonia?

$y^{(1)} = 0$



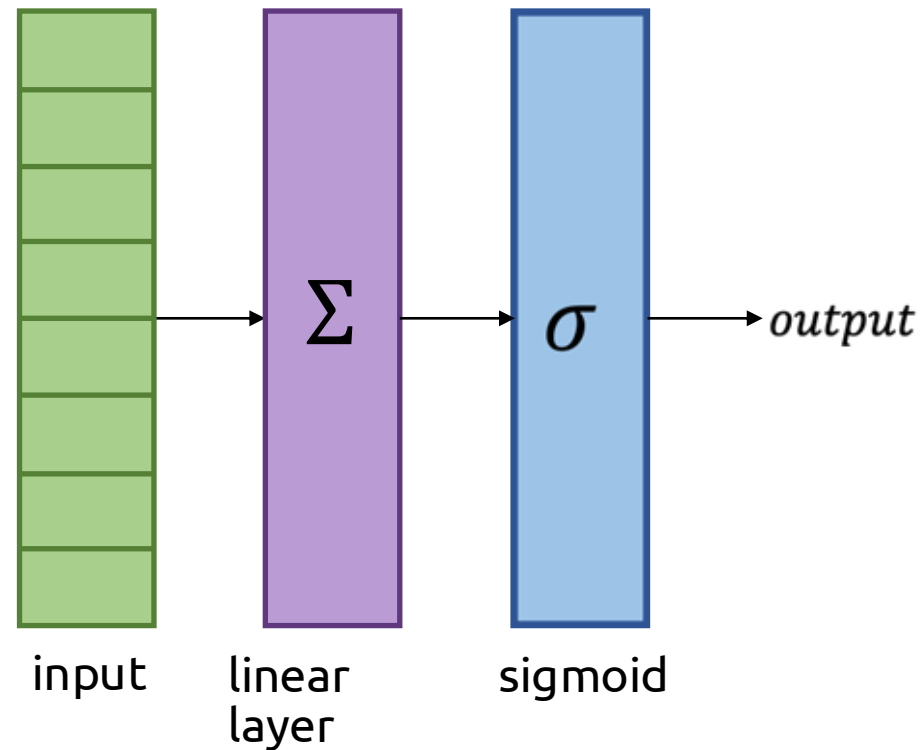
$x^{(2)} =$



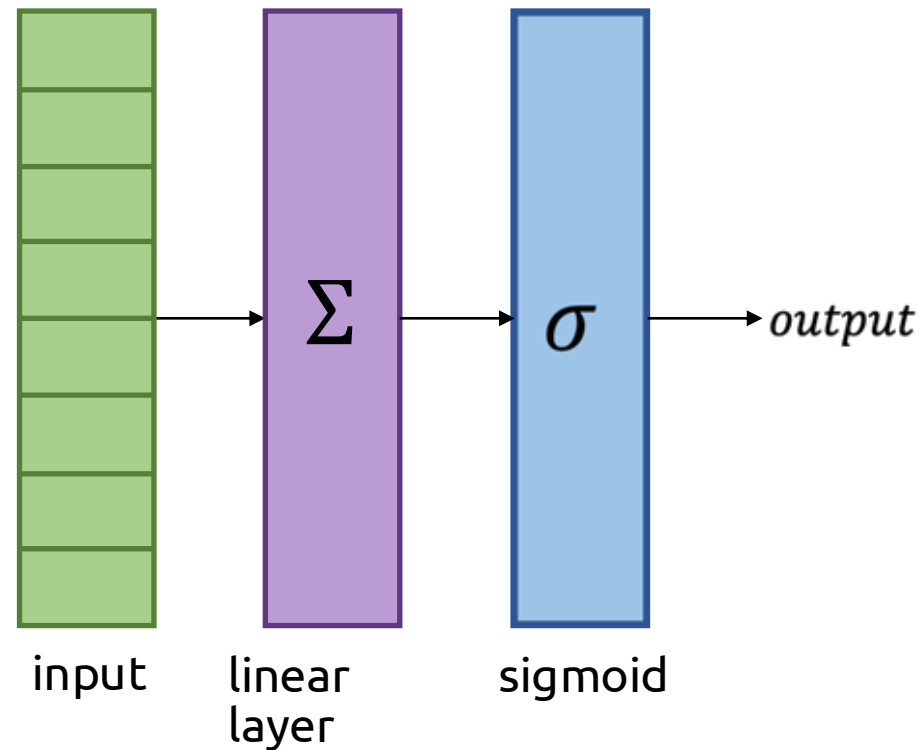
$y^{(2)} = 1$



Single Layer Fully Connected Feed Forward Neural Network



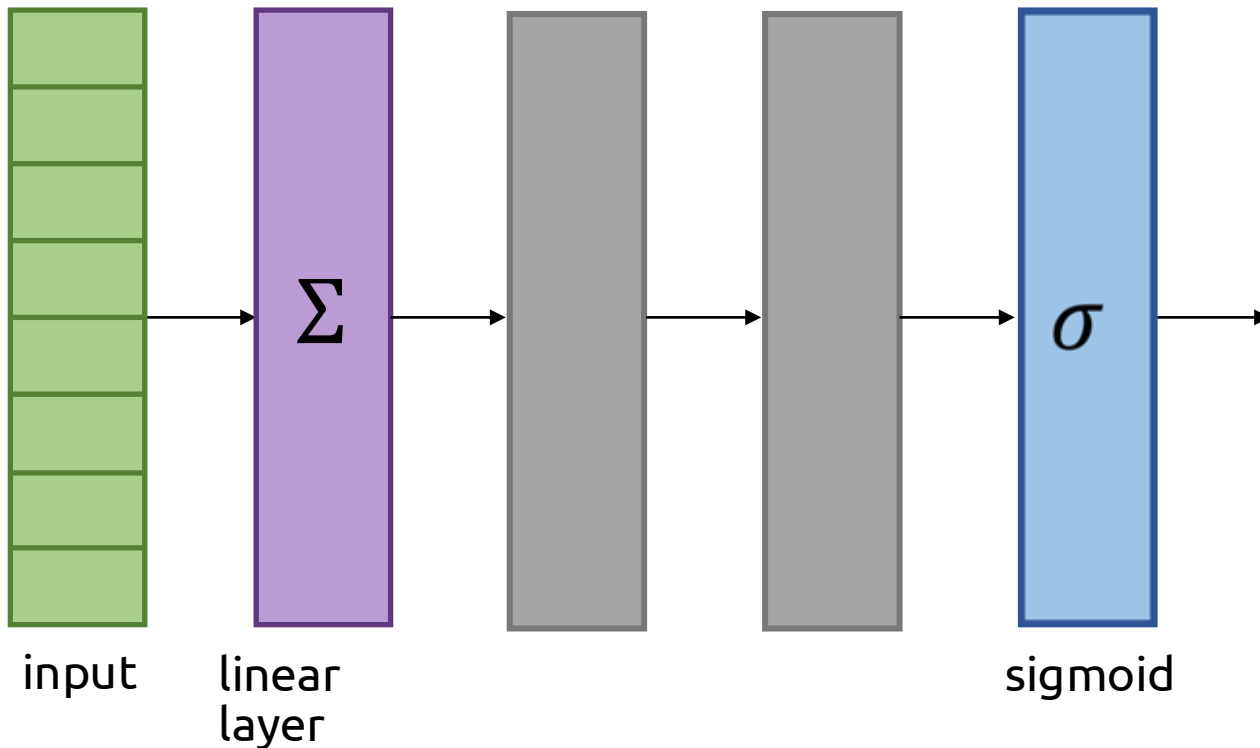
Single Layer Fully Connected Feed Forward Neural Network



How can we do better?

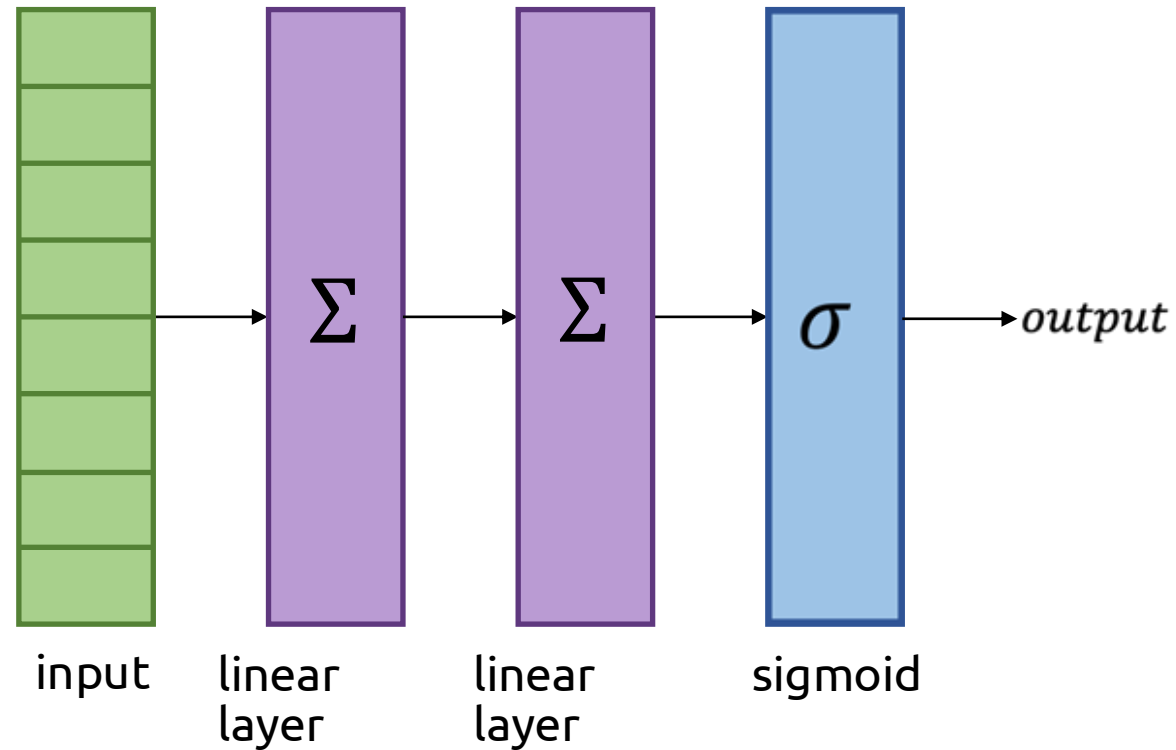
Go deeper!

Multi-layer Neural Networks

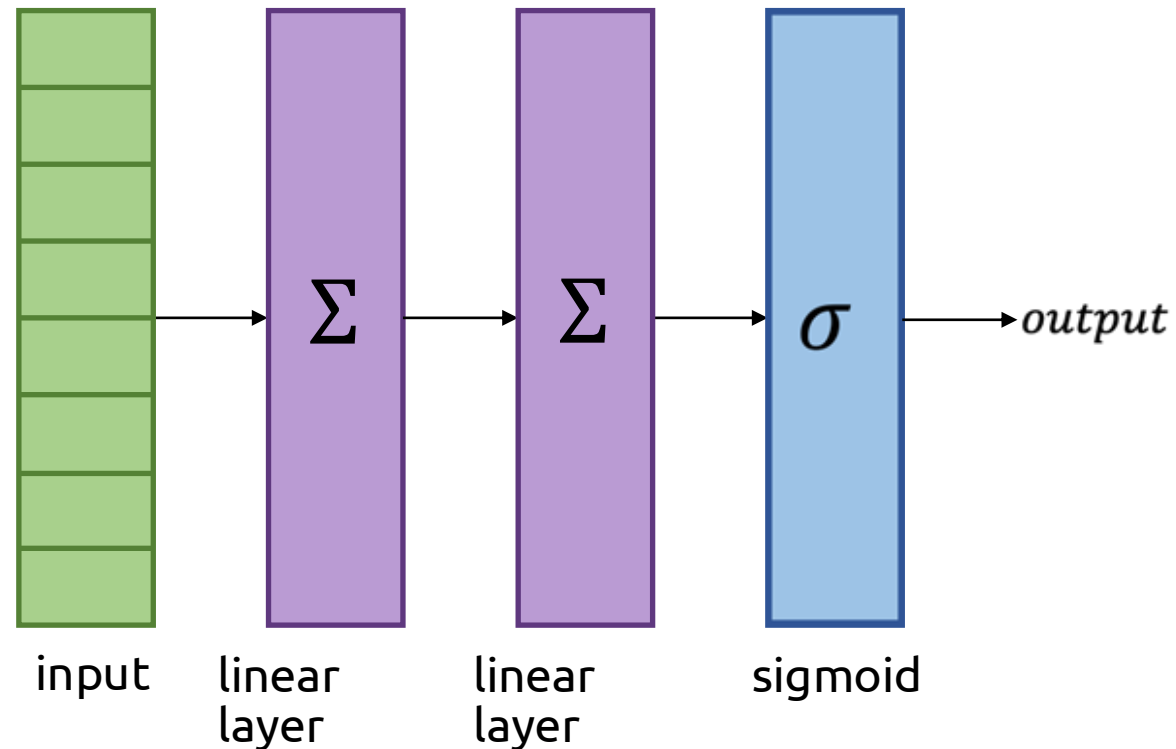


- Each new layer adds another function to the network
 - $f(g(h(\dots z(x) \dots)))$
 - More composed functions $\rightarrow$ can represent more complex computations
- Each new layer has its own tunable parameters
 - More parameters to tune $\rightarrow$ can capture more complex patterns in the data

One Way to Make a Multi-layer Network



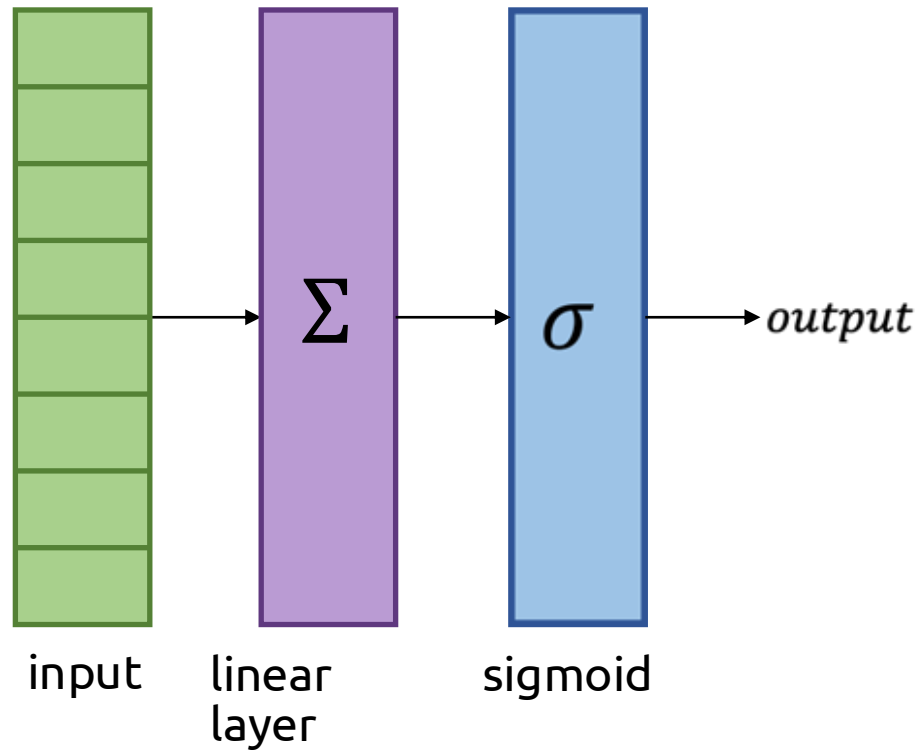
One Way to Make a Multi-layer Network



Obvious idea: just stack more linear layers

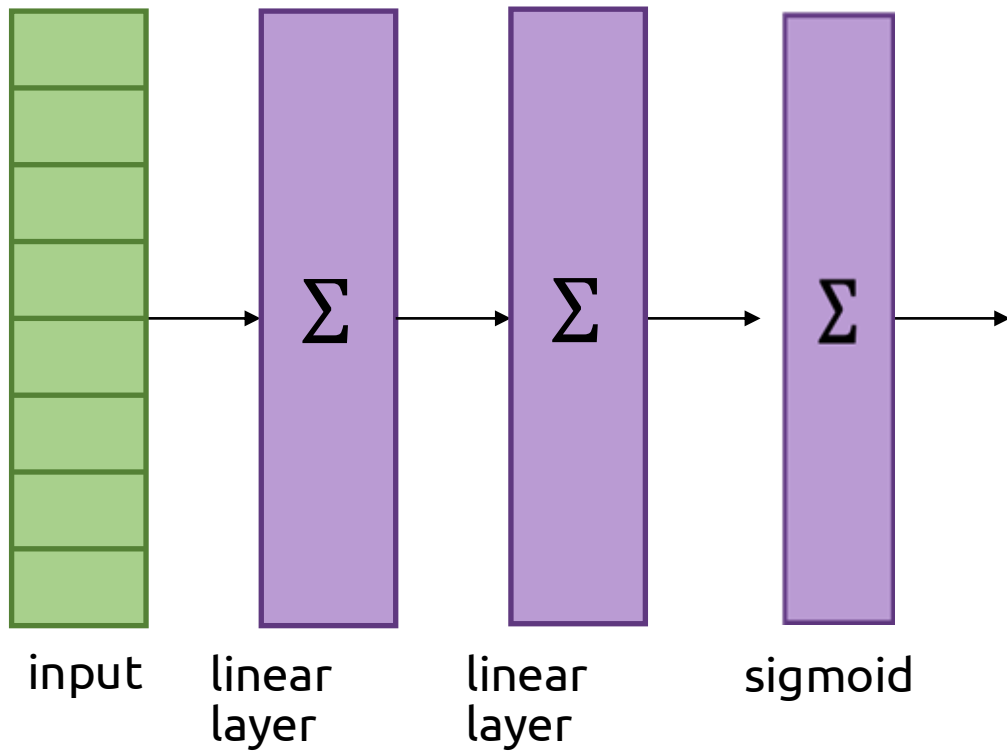
Let's examine the consequences of this design decision...

Single-Layer Network (in math)



$$\sigma \left([w_2 \quad b_2] \begin{bmatrix} x \\ 1 \end{bmatrix} \right)$$

Multi-Layer Network (in math)



$$\sigma \left([w_2 \quad b_2] \left([w_1 \quad b_1] \begin{bmatrix} x \\ 1 \end{bmatrix} \right) \right)$$

Let's simplify this a bit...

Simplifying multi-layer math...

$$\sigma \left([w_2 \quad b_2] \left([w_1 \quad b_1] \begin{bmatrix} x \\ 1 \end{bmatrix} \right) \right)$$

Simplifying multi-layer math...

$$\sigma \left([w_2 \quad b_2] \left([w_1 \quad b_1] \begin{bmatrix} x \\ 1 \end{bmatrix} \right) \right)$$

Apply associativity...

$$\sigma \left(([w_2 \quad b_2][w_1 \quad b_1]) \begin{bmatrix} x \\ 1 \end{bmatrix} \right)$$

Multiply the matrices...

$$\sigma \left([w_{12} \quad b_{12}] \begin{bmatrix} x \\ 1 \end{bmatrix} \right)$$

Simplifying multi-layer math...

$$\sigma \left([w_2 \quad b_2] \left([w_1 \quad b_1] \begin{bmatrix} x \\ 1 \end{bmatrix} \right) \right)$$

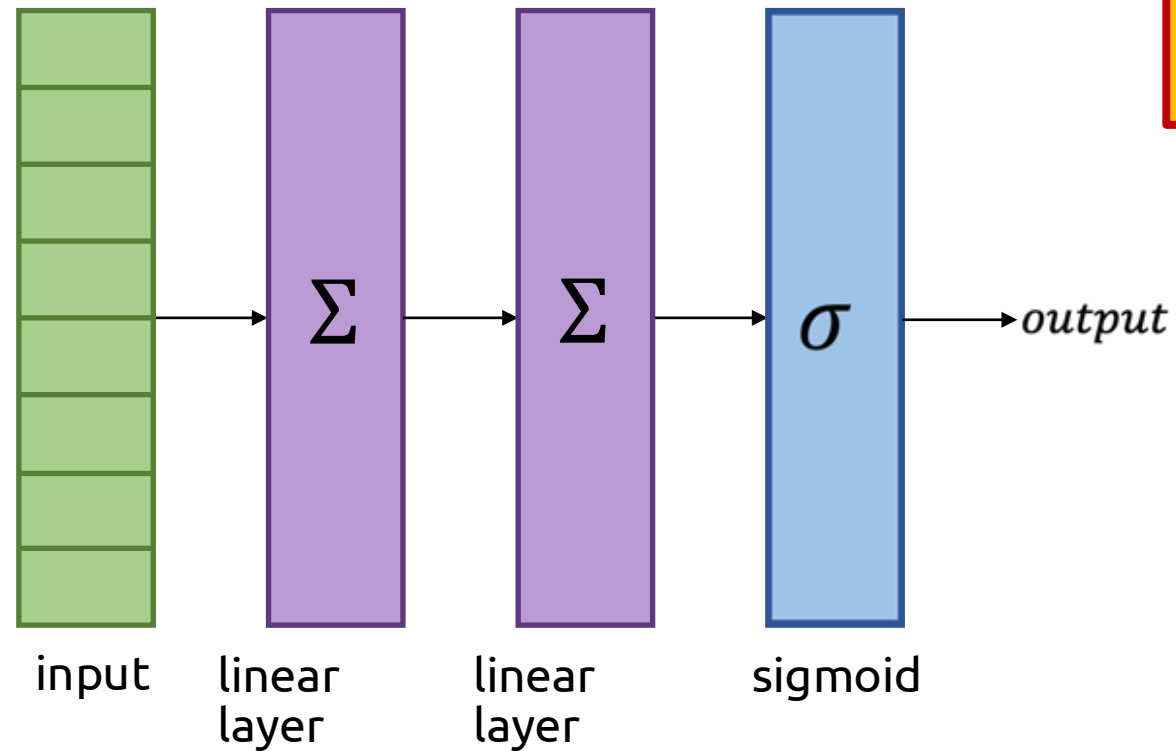
Apply associativity...

$$\sigma \left(([w_2 \quad b_2][w_1 \quad b_1]) \begin{bmatrix} x \\ 1 \end{bmatrix} \right)$$

Multiply the matrices...

Same as a one-layer network $\longrightarrow \sigma \left([w_{12} \quad b_{12}] \begin{bmatrix} x \\ 1 \end{bmatrix} \right)$

Takeaway: Stacking Linear Layers Isn't Enough



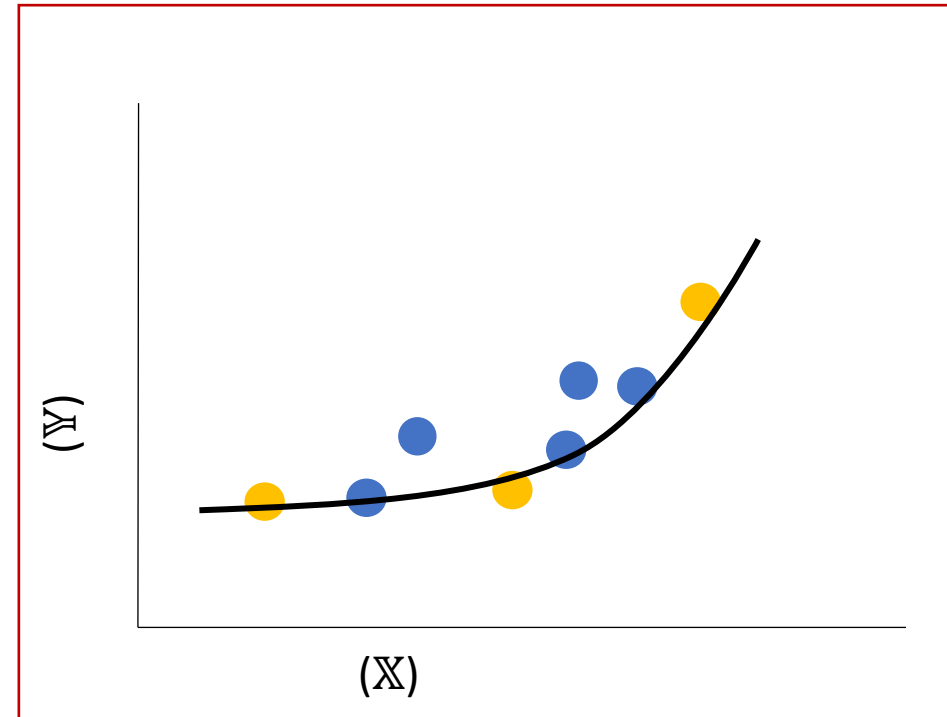
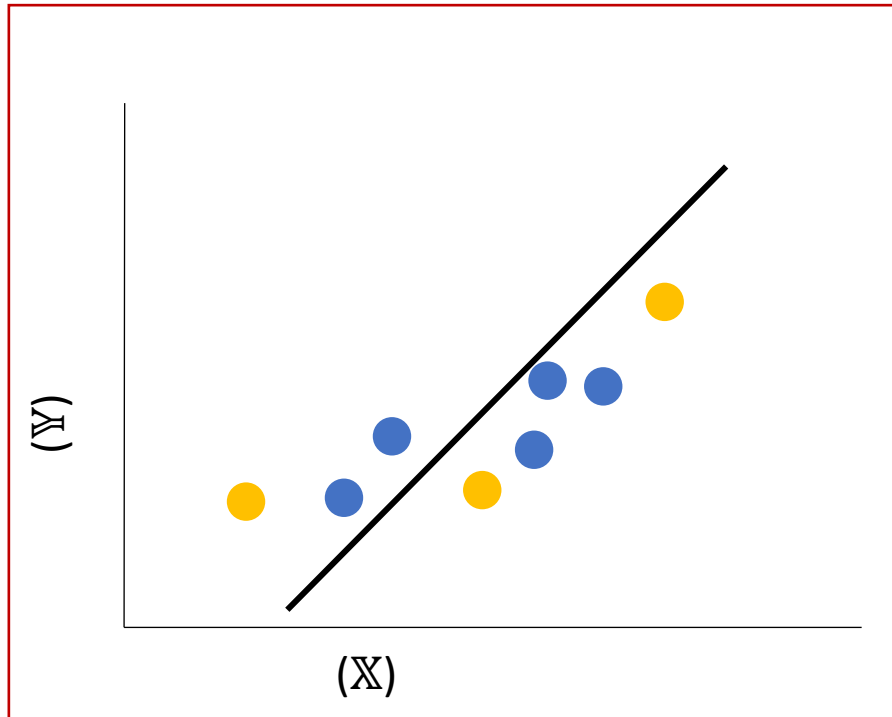
Combination of linear functions is another linear function.

Why is this a problem?



Linear functions may not be sufficient

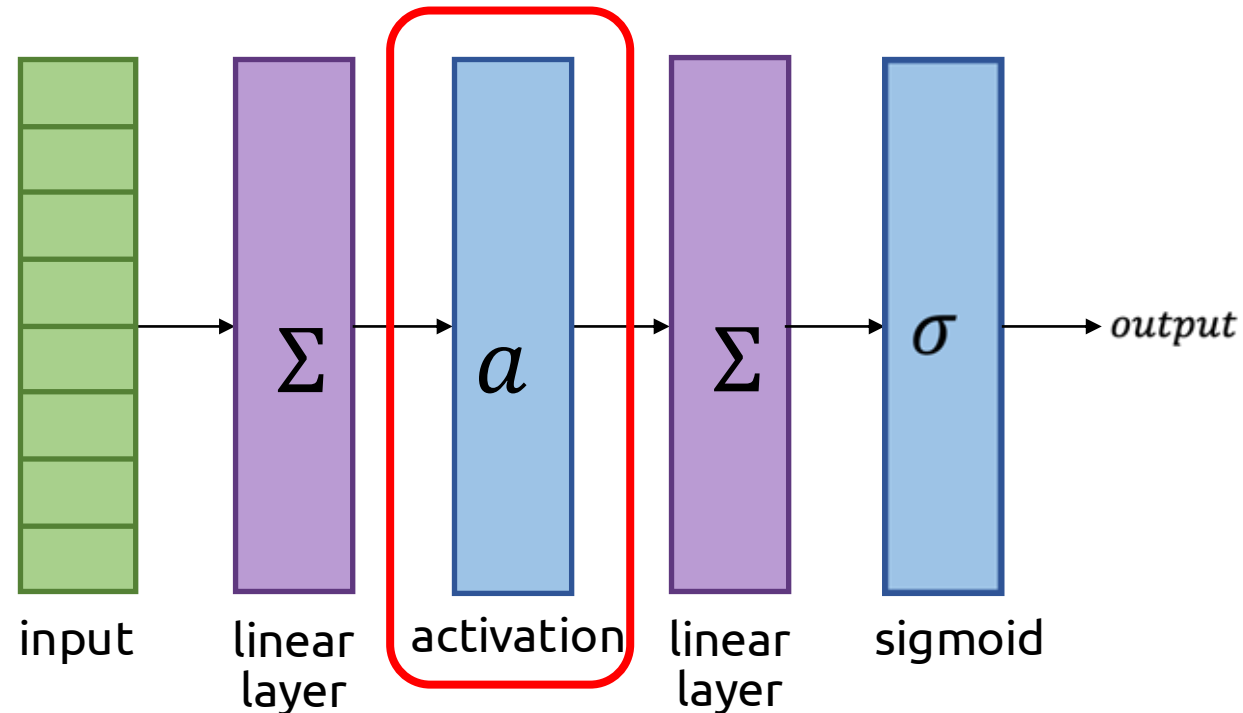
- Root cause of our problem: a composition of linear functions is still linear



Adding non-linearity using
activation functions

Incorporate non-linearity - Activation Functions

- Root cause of our problem: a composition of linear functions is still linear
- Need some kind of nonlinear function between each linear layer.
- Called an **activation function**
 - Origin of the name: a neuron “activates” if it gets enough electrochemical input

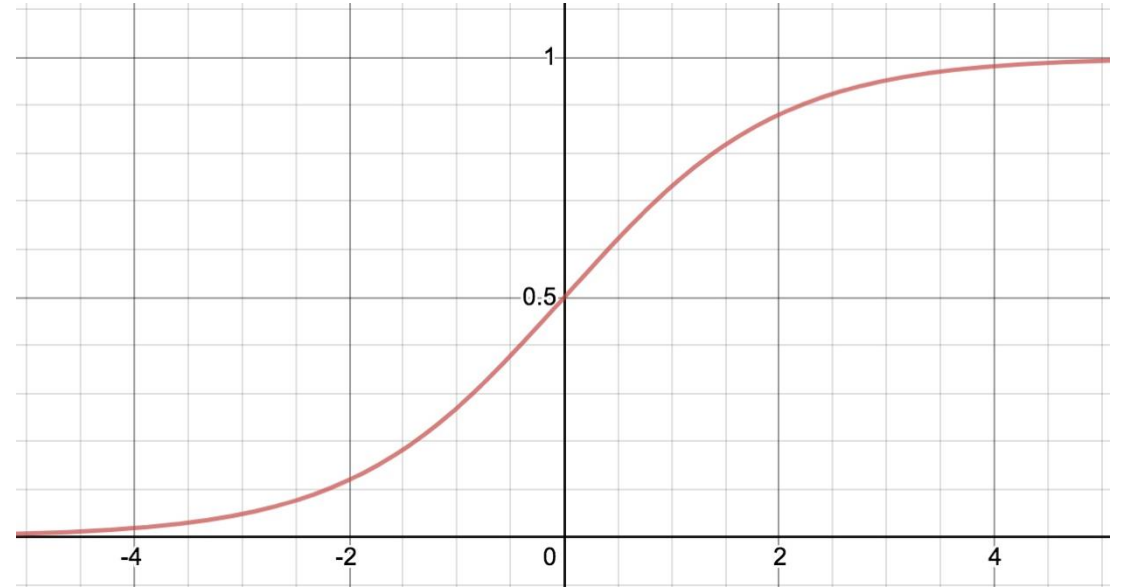


The Sigmoid Activation Function

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

The Sigmoid Activation Function

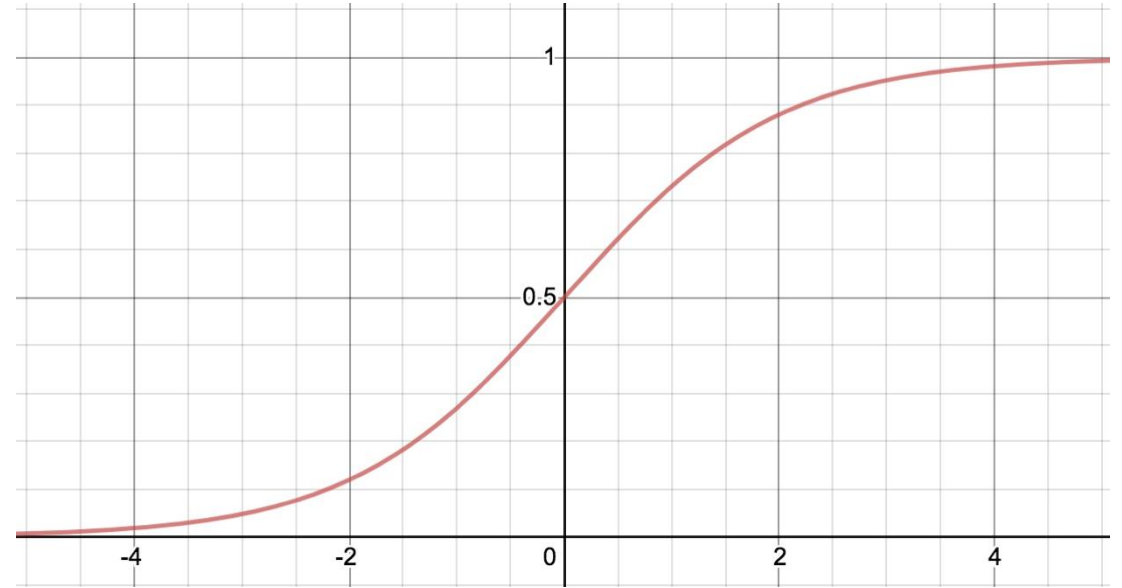
- Historically very popular activation function
- **Takes real value and squashes it to range between 0 and 1**
 - i.e. $\sigma(x): \mathbb{R} \rightarrow (0, 1)$



$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

The Sigmoid Activation Function

- Large negative numbers become 0 and large positive numbers become 1
- **Bounded:** guarantees gradient cannot grow without bound!!



$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Another “Sigmoidal” function: Tanh

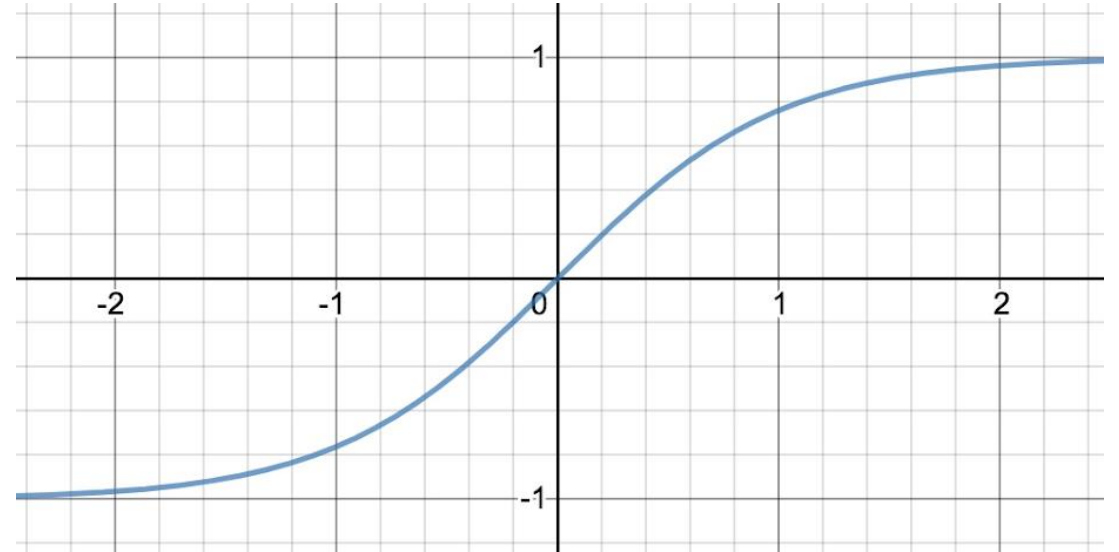
$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

The hyperbolic tangent function

Tanh

- Output range: $[-1, 1]$
 - Versus sigmoid $[0, 1]$
- Somewhat desirable property of keeping the signal that passes through the network “centered” around zero.

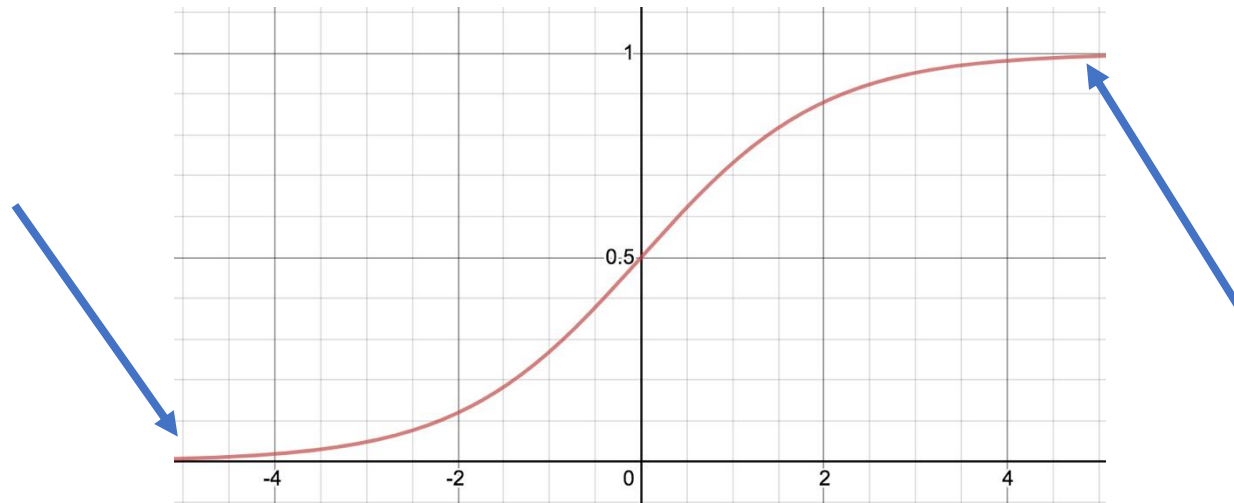
Do you see any issues with these functions?
(Think about the gradients!)



$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

But there is an issue...

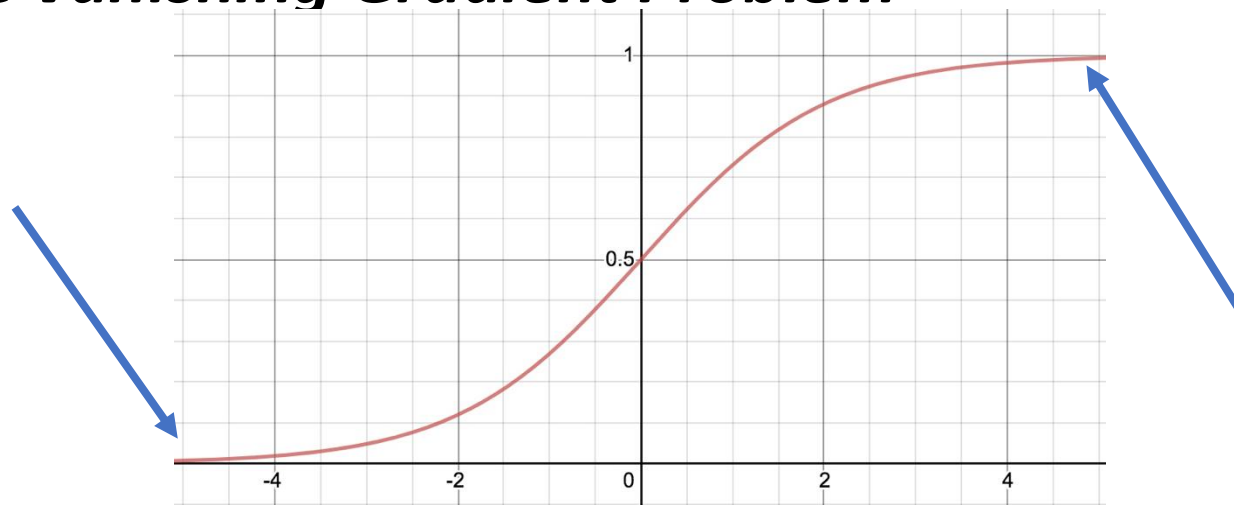
- The bounded-ness of these functions is a double-edged sword
 - Why? Being bounded means that the function has ***asymptotes***, which have zero derivative in the limit.



$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

But there is an issue...

- So, our derivatives don't grow out of control...
- ...but the price we pay is that they approach zero, and ***the network stops learning***
- Known as the ***Vanishing Gradient Problem***



$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Consequences of Vanishing Gradients

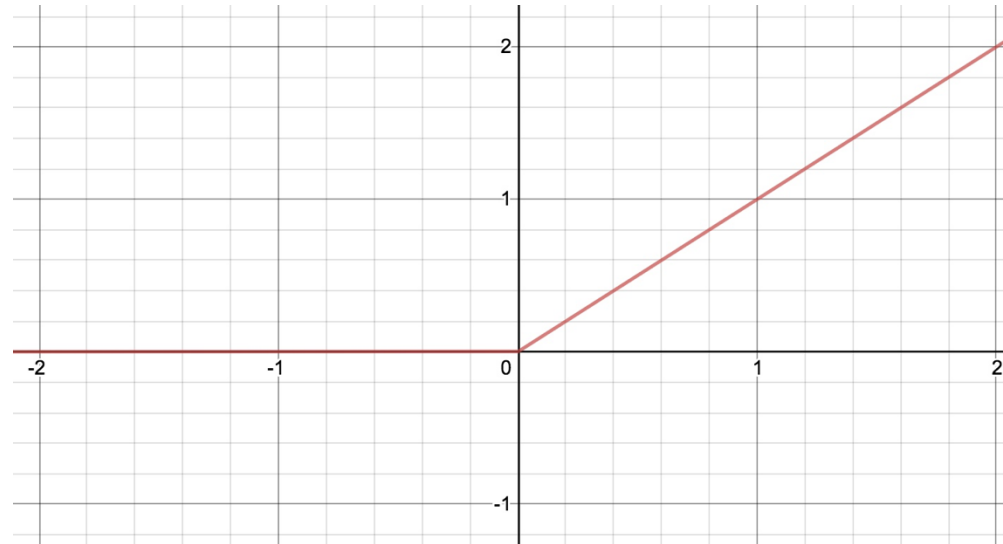
- Problem is exacerbated by stacking multiple layers (gradients shrink more the deeper you go)
- Led to the belief that in practice, neural nets could only ever be a few layers deep...

Any questions?



Enter the *Rectifier Function*

- Nonlinear — cannot be represented as: $a(x) + b$

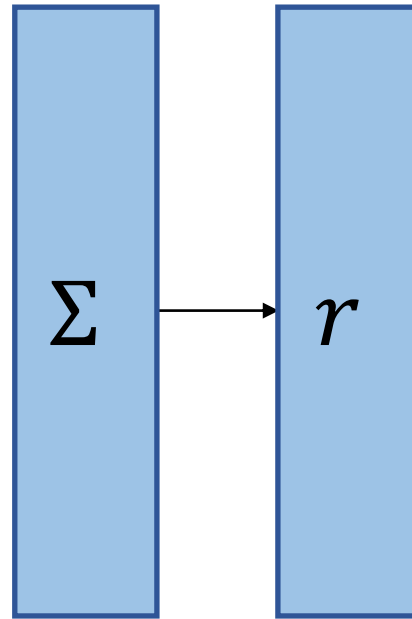


$$f(x) = \begin{cases} x, & x > 0 \\ 0, & \text{else} \end{cases}$$

More commonly known as ReLU

- ***Rectified Linear Unit***

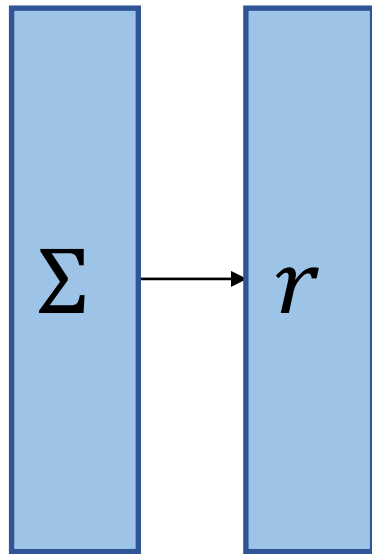
- Technically: Linear layer followed by the rectifier function
- But in most contexts, you will see the rectifier function called “ReLU”



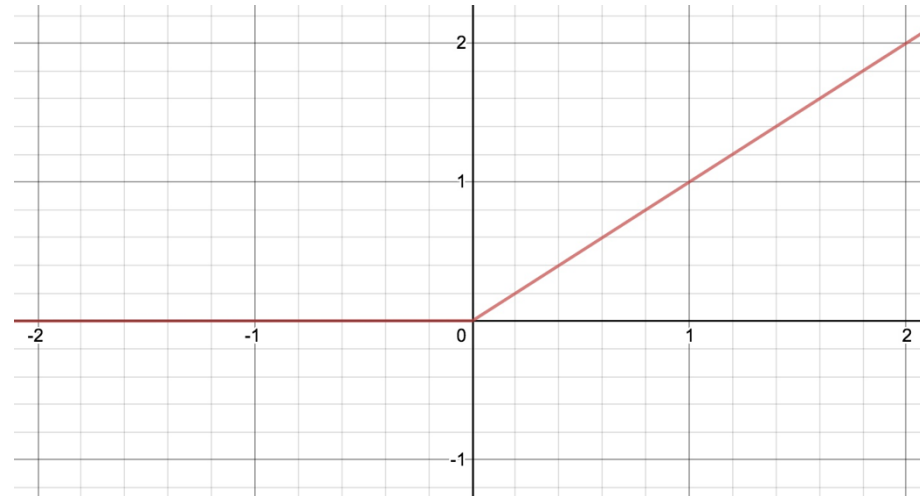
Rectified Linear Unit

Advantages of ReLU

- Does not suffer from vanishing **or** exploding gradients!
- Super computationally efficient (avoids the exp calls in sigmoid/tanh)
- Most popular, de-facto 'standard' activation function



Rectified Linear Unit



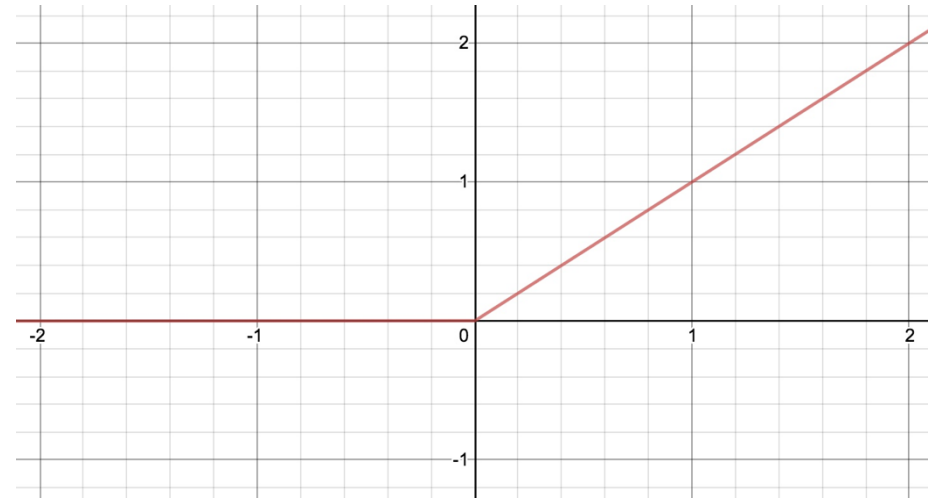
$$\text{ReLU}(x) = \max(0, x)$$

$$f(x) = \begin{cases} x, & x > 0 \\ 0, & \text{else} \end{cases}$$

But not even ReLU is perfect...

- We said that the zero-derivative asymptotes of sigmoid were a problem...

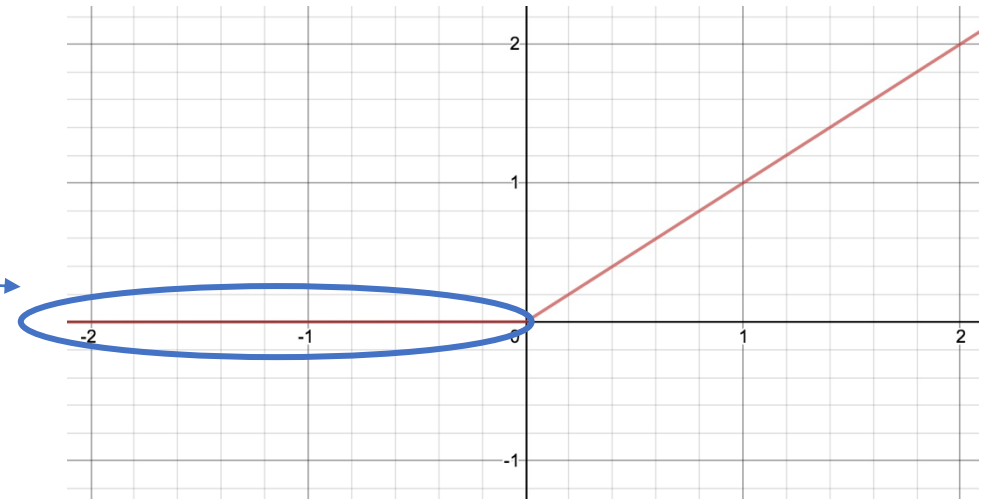
Do we see any issues here?



$$f(x) = \begin{cases} x, & x > 0 \\ 0, & \text{else} \end{cases}$$

But not even ReLU is perfect...

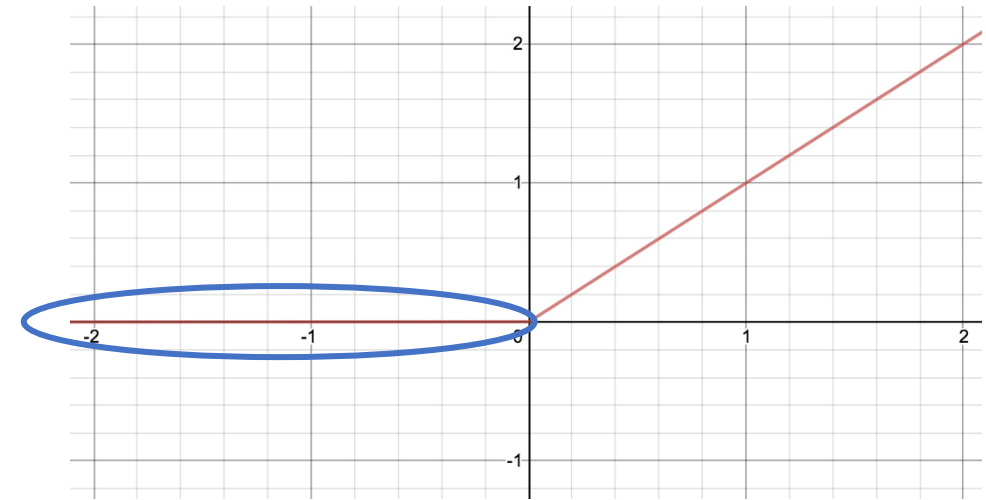
- We said that the zero-derivative asymptotes of sigmoid were a problem...
- Check out this huge zero-derivative region
- Effectively: layers that feed into this activation ***don't learn anything*** if they feed negative values



$$f(x) = \begin{cases} x, & x > 0 \\ 0, & \text{else} \end{cases}$$

But not even ReLU is perfect...

- Not such a big deal if the previous layer just occasionally produces negative values
 - Some people even claim this as a “feature,” in that the resulting ‘sparse activations’ in the network more closely resemble what the human brain does
- But what if the previous layer ***always*** produces negative values?
- Is this even possible?



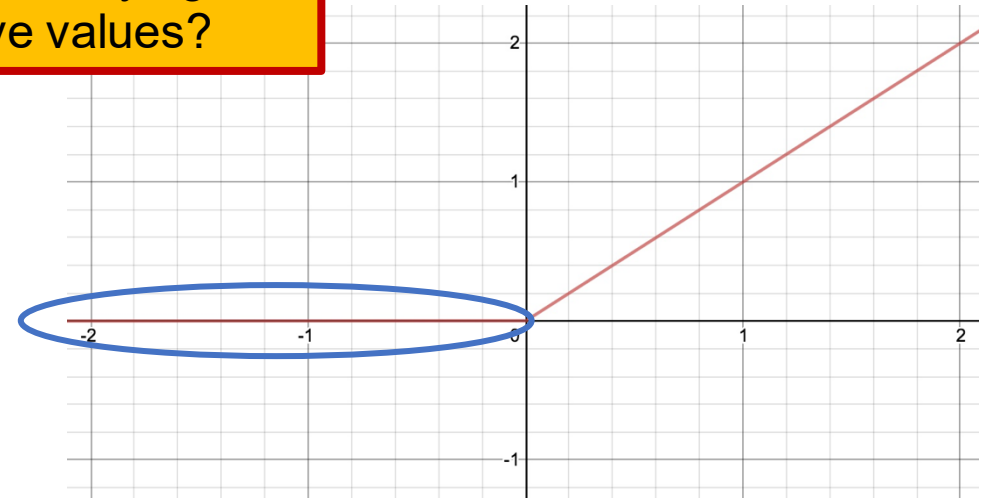
$$f(x) = \begin{cases} x, & x > 0 \\ 0, & \text{else} \end{cases}$$

But not even ReLU is perfect...

- The value fed into ReLU:

- $l_j = \sum_k W_{j,k} x_k + b_j$

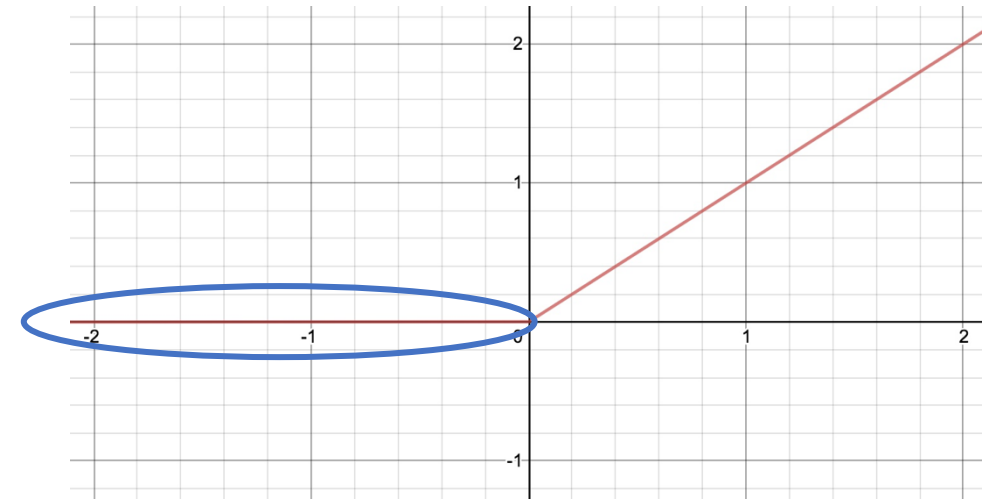
Thinking activity: How could we always get negative values?



$$f(x) = \begin{cases} x, & x > 0 \\ 0, & \text{else} \end{cases}$$

But not even ReLU is perfect...

- The value fed into ReLU:
 - $l_j = \sum_k W_{j,k} x_k + b_j$
- If our inputs x_k are bounded (e.g. $[0,1]$), then the following is possible:
 - The weights have small magnitude
 - The bias is a large negative number
- In this case, l_j will always be negative!



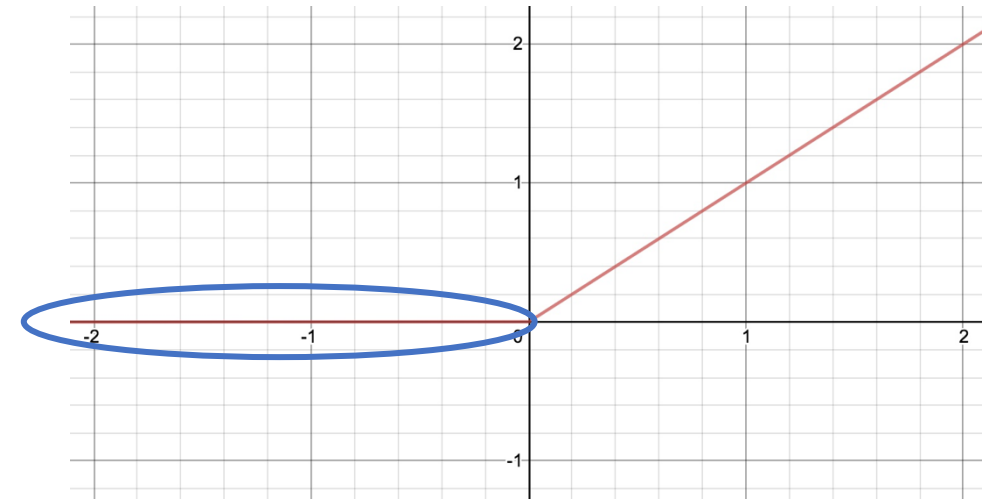
$$f(x) = \begin{cases} x, & x > 0 \\ 0, & \text{else} \end{cases}$$



Known as the **Dead ReLU problem**

But not even ReLU is perfect...

- Does this ever happen in practice?
 - Yes! A large gradient update can 'accidentally' knock the parameters into a state where this happens.
 - [Known cases](#) where as much as 40% of the network suffers from this



$$f(x) = \begin{cases} x, & x > 0 \\ 0, & \text{else} \end{cases}$$

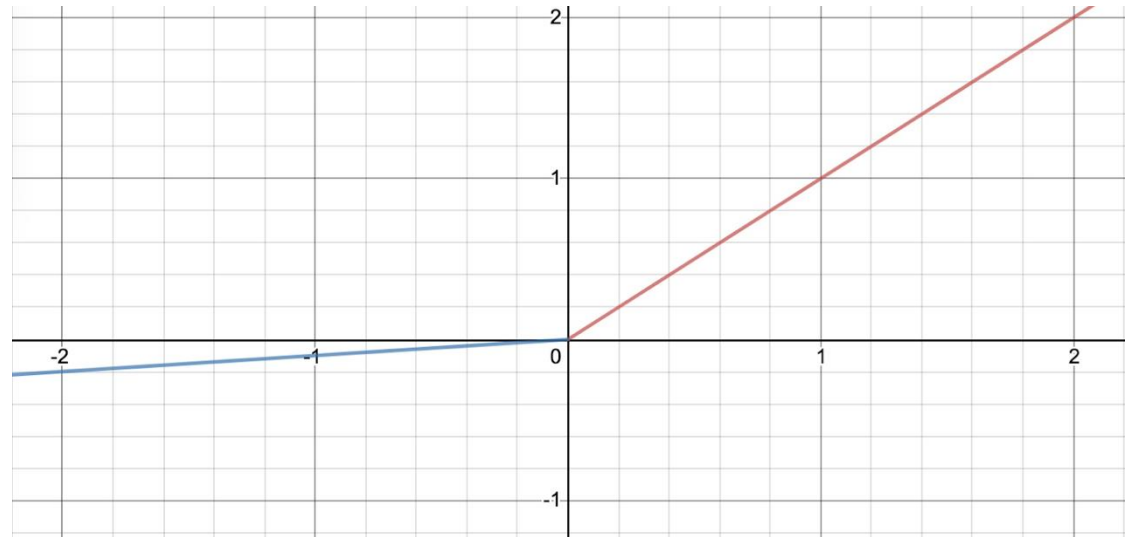


Known as the **Dead ReLU problem**

Leaky ReLU

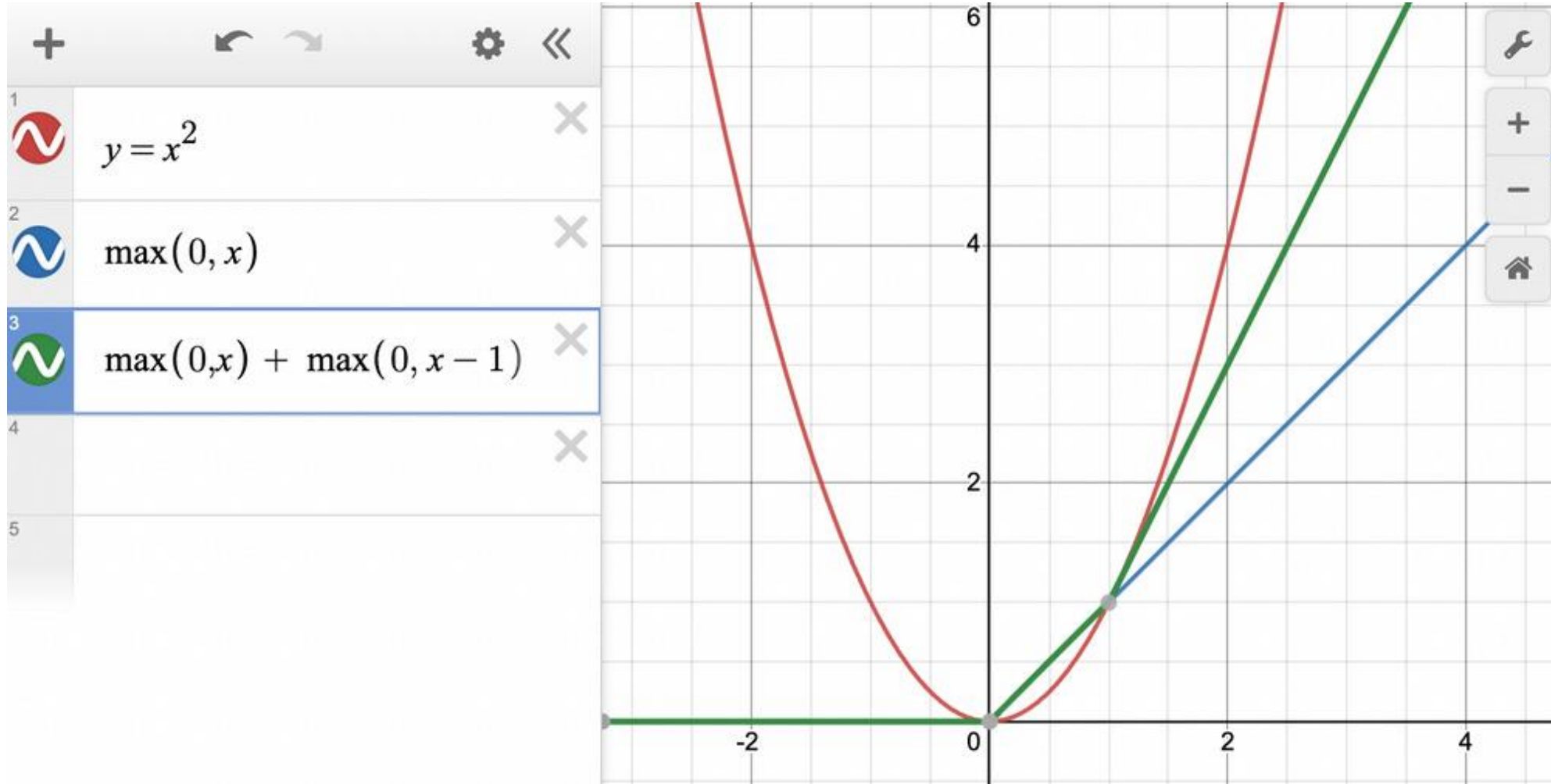
- Fix — we give a tiny positive slope for negative inputs
- Some activation “leaks” through the barrier

$$f(x) = \begin{cases} x, & x > 0 \\ ax, & \text{else} \end{cases}$$



$$\text{LeakyReLU}(x) = \max(0, x) + a * \min(0, x)$$

Non-linearity as piecewise-linearity



Refer: <https://www.blog.dailydoseofds.com/p/a-visual-and-intuitive-guide-to-what/>

Other Activation Functions

Why use other activation functions?

Softplus

```
CLASS torch.nn.Softplus(beta=1, threshold=20) [SOURCE]
```

Applies the element-wise function:

$$\text{Softplus}(x) = \frac{1}{\beta} * \log(1 + \exp(\beta * x))$$

SoftPlus is a smooth approximation to the ReLU function and can be used to constrain the output of a machine to always be positive.

For numerical stability the implementation reverts to the linear function for inputs above a certain value.

Hardshrink

```
CLASS torch.nn.Hardshrink(lambda=0.5) [SOURCE]
```

Applies the hard shrinkage function element-wise:

$$\text{HardShrink}(x) = \begin{cases} x, & \text{if } x > \lambda \\ x, & \text{if } x < -\lambda \\ 0, & \text{otherwise} \end{cases}$$

LogSigmoid

```
CLASS torch.nn.LogSigmoid [SOURCE]
```

Applies the element-wise function:

$$\text{LogSigmoid}(x) = \log\left(\frac{1}{1 + \exp(-x)}\right)$$

CELU

```
CLASS torch.nn.CELU(alpha=1.0, inplace=False) [SOURCE]
```

Applies the element-wise function:

$$\text{CELU}(x) = \max(0, x) + \min(0, \alpha * (\exp(x/\alpha) - 1))$$

More details can be found in the paper [Continuously Differentiable Exponential Linear Units](#).

Parameters

- **alpha** – the α value for the CELU formulation. Default: 1.0
- **inplace** – can optionally do the operation in-place. Default: `False`

Shape:

- Input: $(N, *)$ where $*$ means, any number of additional dimensions
- Output: $(N, *)$, same shape as the input

Great PyTorch documentation [here!](#)

Reasons to use other activation functions

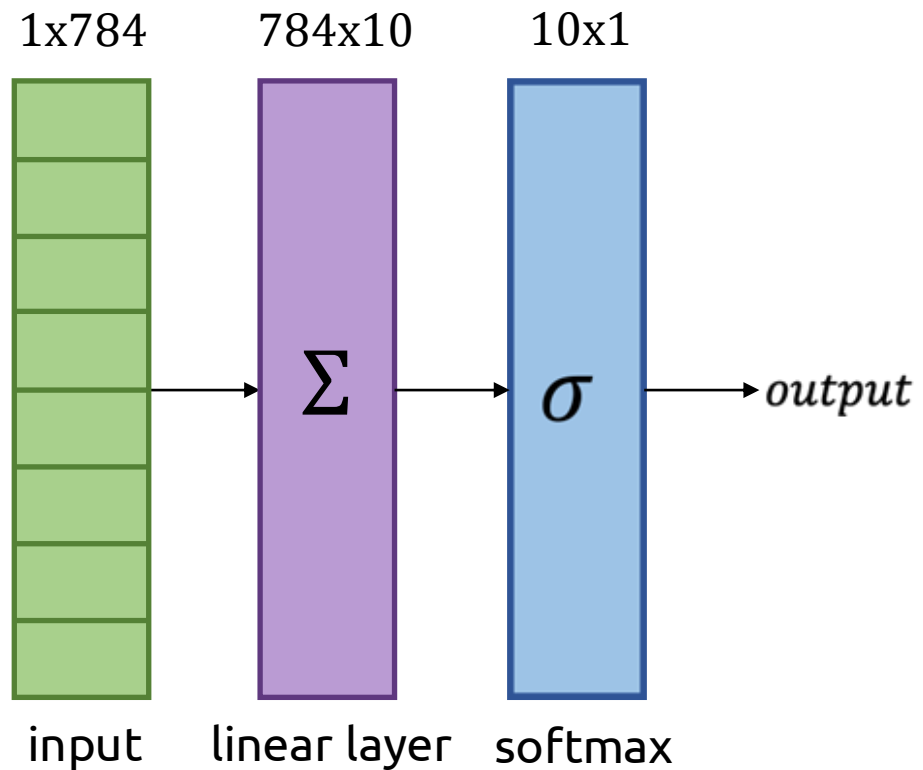
- Bounding network outputs to a particular range
 - Tanh: $[-1, 1]$
 - Sigmoid: $[0, 1]$
 - Softplus: $[0, \infty]$
- Example: Predicting a person's age from other biological features
 - Age is a strictly positive quantity
 - We can help our network learn by restricting it to output only positive numbers
 - Use a **Softplus activation** on the output

Any questions?



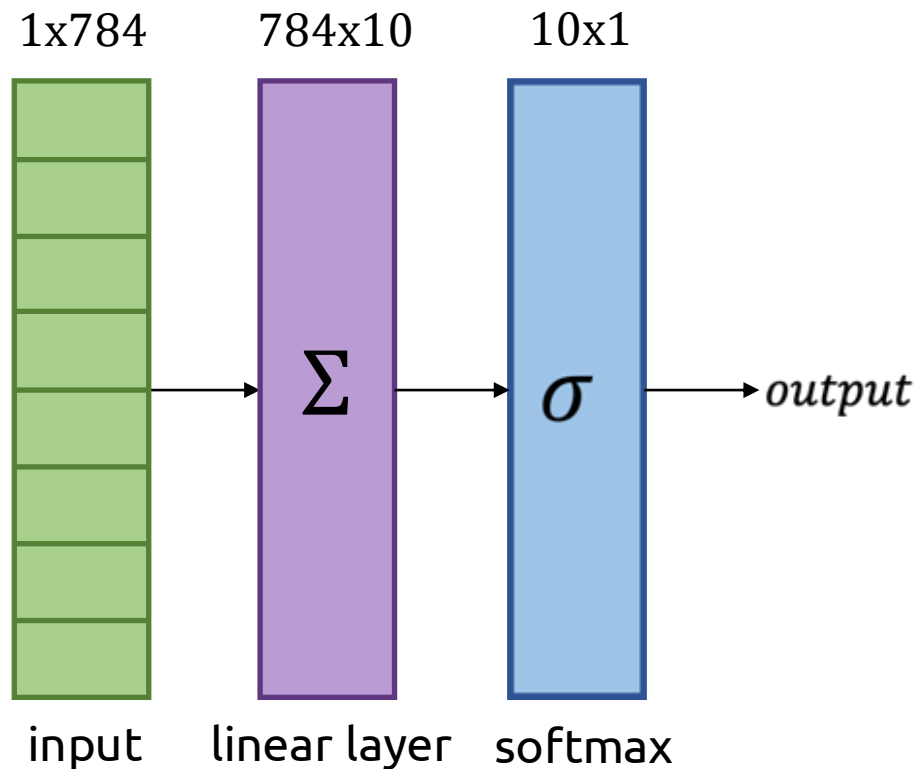
Building a multi-layer network

- Previously:

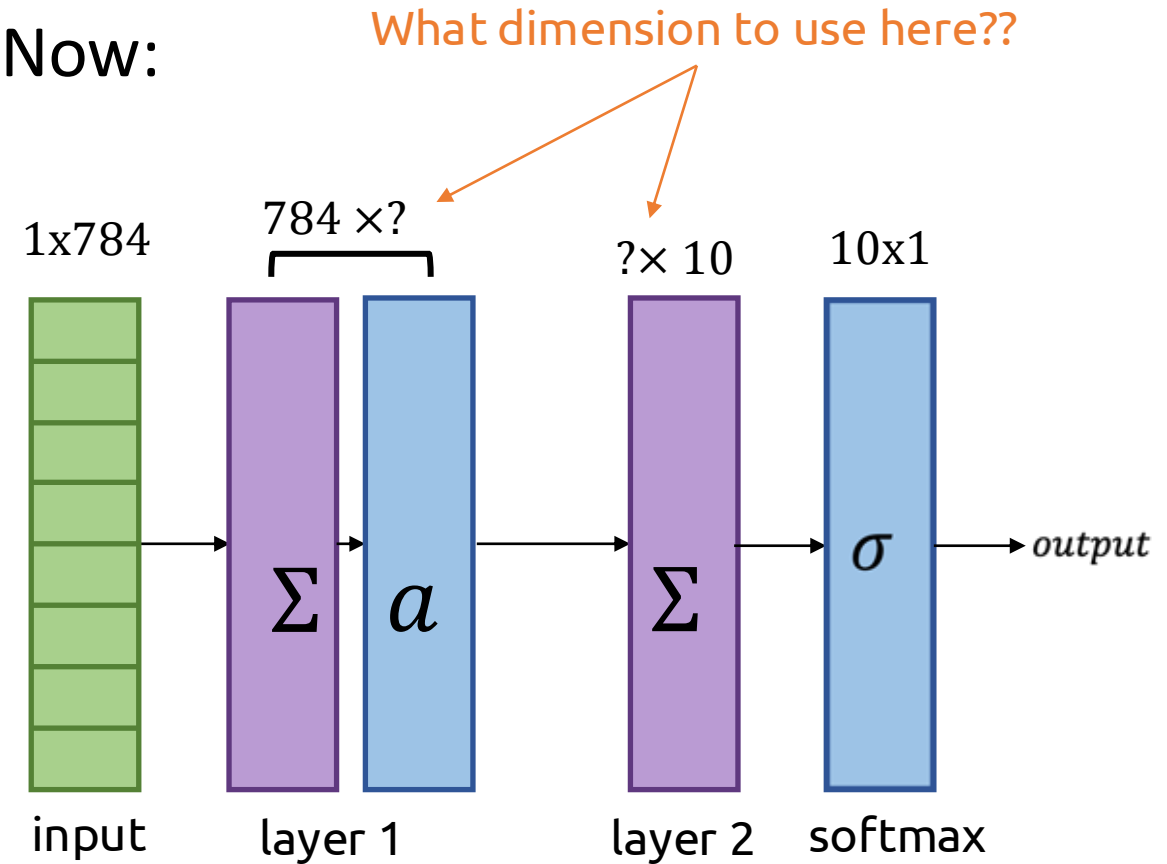


Consequences of adding activation layers

- Previously:

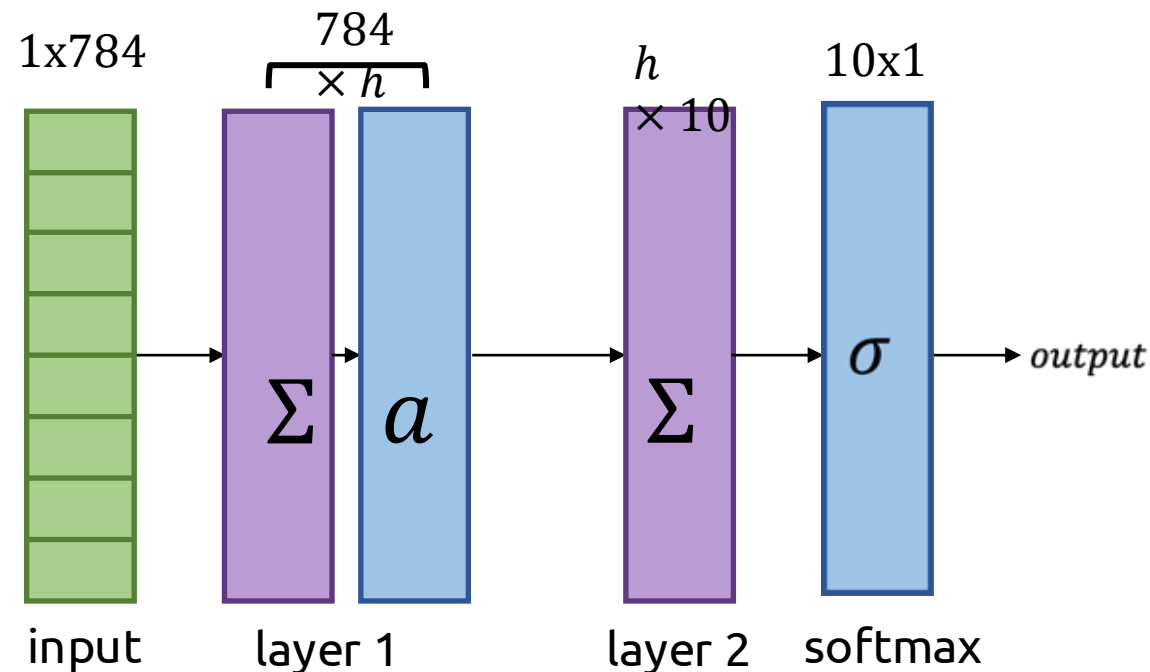


- Now:



“Hidden Layers”

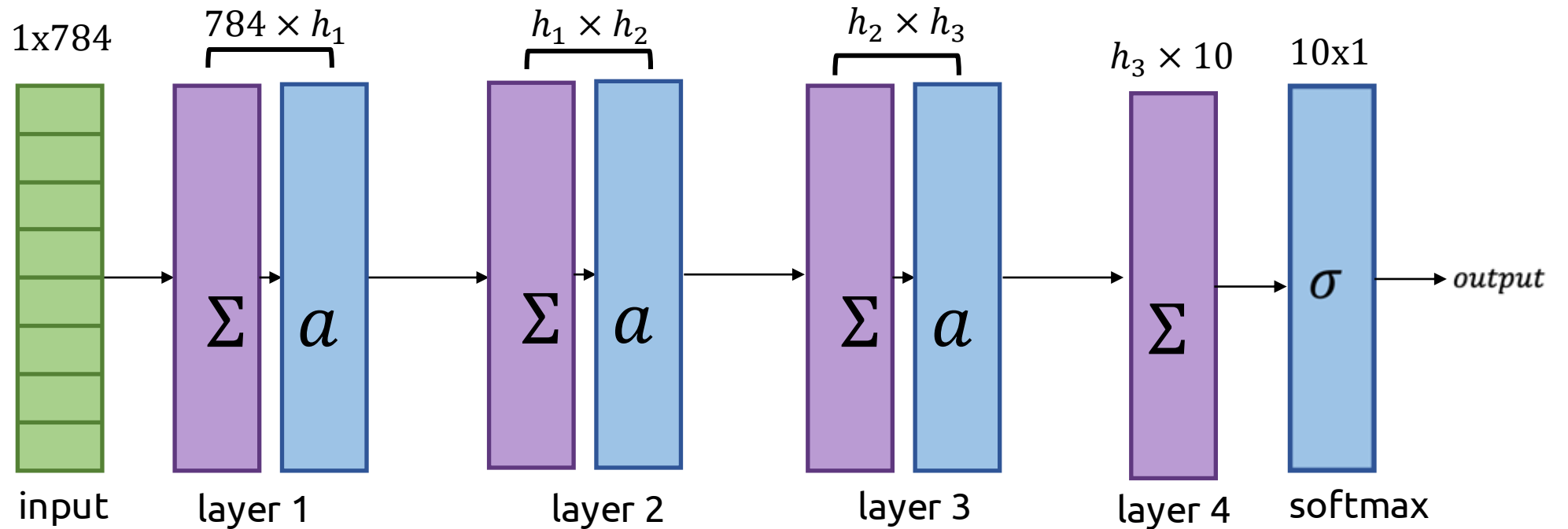
- The output of a function that doesn't feed into the output layer (like softmax) is called a ***hidden layer***
- Have to set the size h of these hidden layers
- More linear units $\rightarrow$ more hidden layer sizes



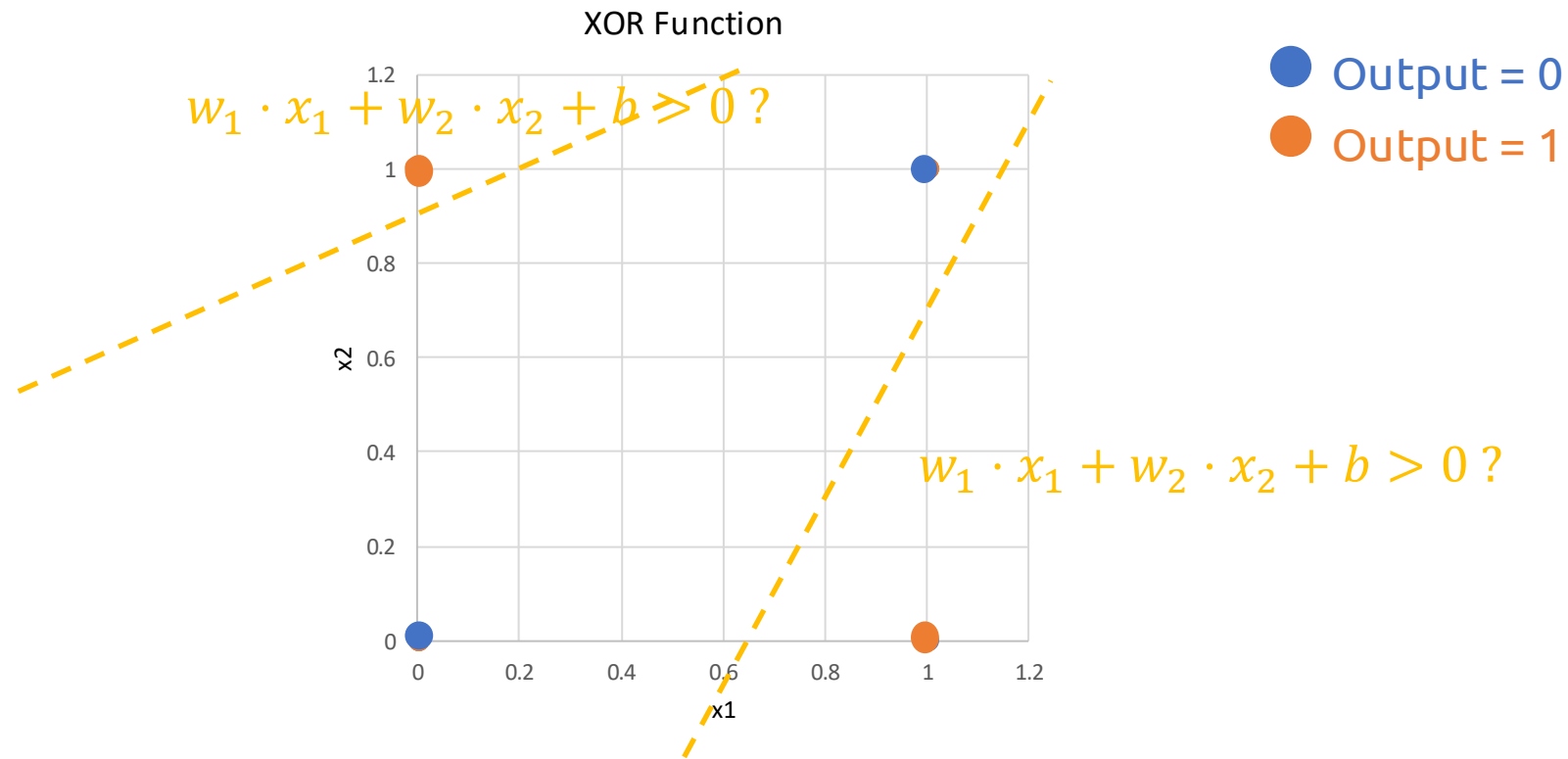
Hyperparameters

- Hidden layer sizes are a ***hyperparameter*** — configuration external to model, value usually set before training begins
 - Number of epochs, batch size, etc.
 - Contrast this with a learnable ***parameter***, we keep talking about
- Rule of thumb
 - Start out making hidden layers the same size as the input
 - Then, tweak it to see the effect
- There are more principled (and time-consuming) ways to set them
 - Grid search, random search, Bayesian optimization...
 - See [here](#) for an overview and more references

What a multi-layer neural network could look like?



Can a multi-layer network learn XOR?



Class activity

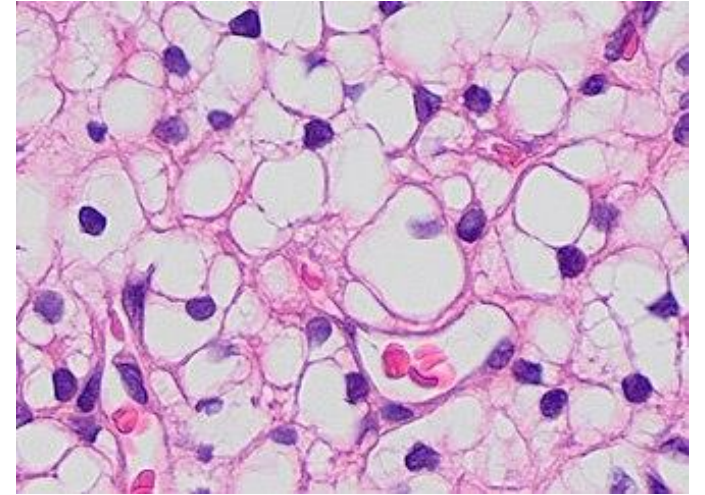
[Activation Function Demo](#)

What kind of datasets CNNs
are popularly applied to?

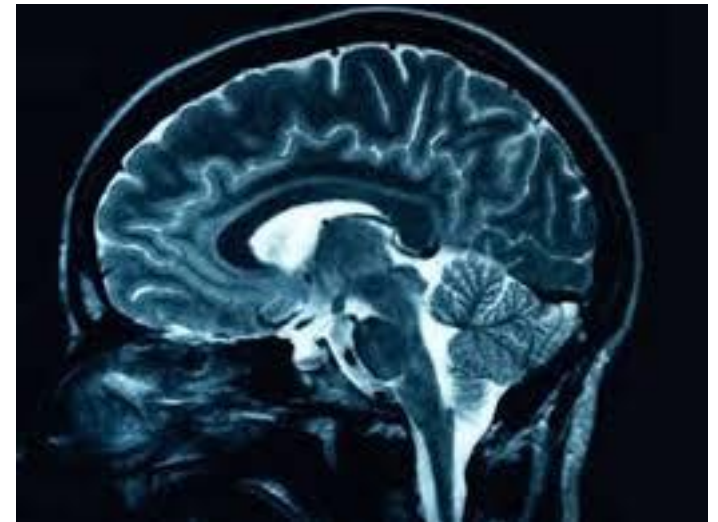
Convolution and CNNs



X-ray



Histopathology



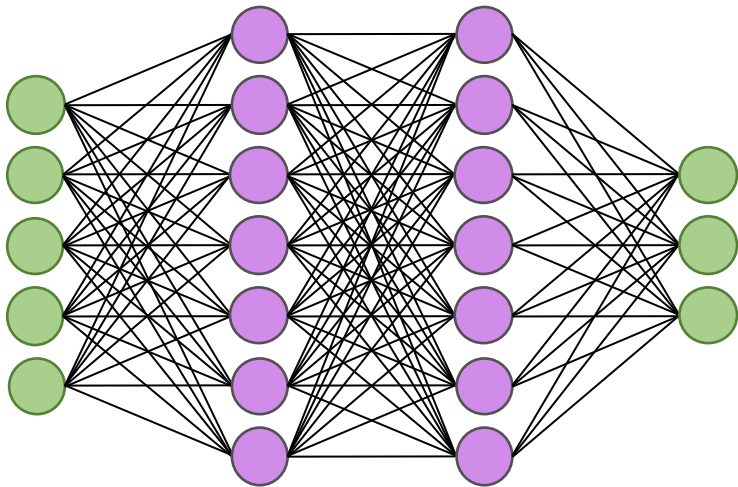
Magnetic Resonance Imaging (MRI)

Images!

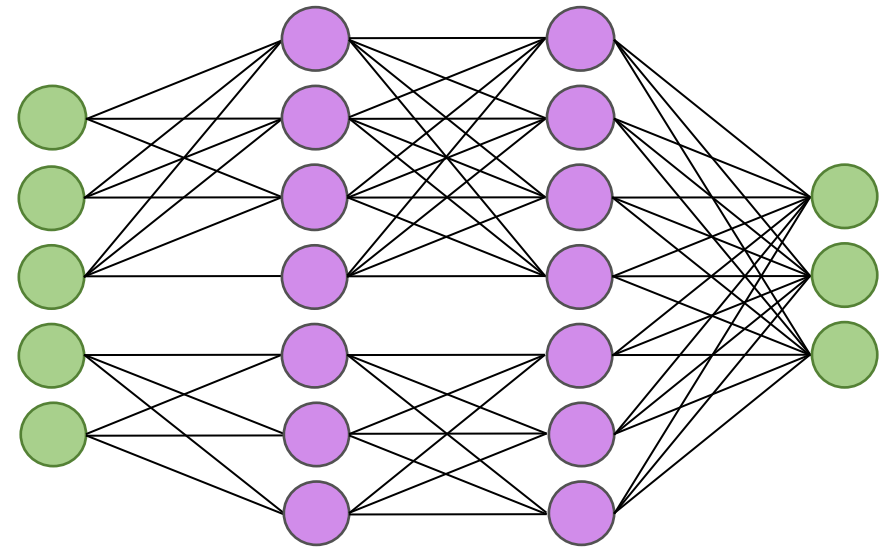


Does a network have to be fully connected?

Fully Connected



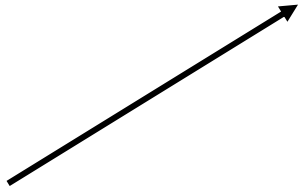
Partially Connected?



Why would you ever want to do this?

Partially Connected Networks?

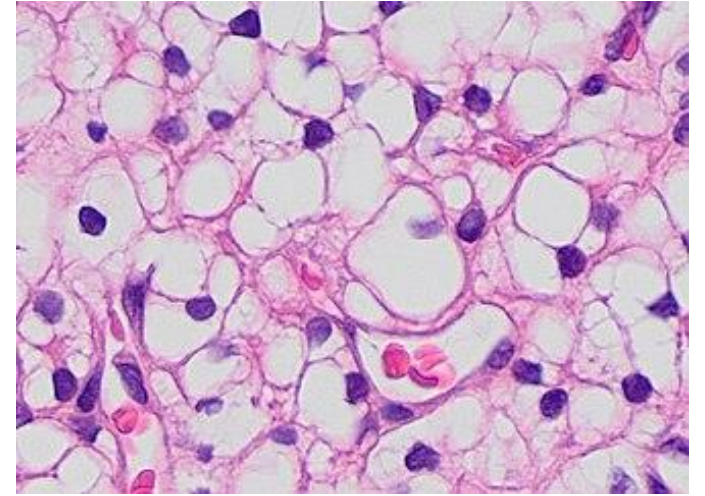
- Fewer connections == Worse results? ...right?
- Advantages of Partial Connections
 - Fewer connections → fewer weights to learn
 - Faster training; more compact models; better generalization performance
 - Can design connectivity pattern that exploits knowledge of the data (like connecting patterns in features)



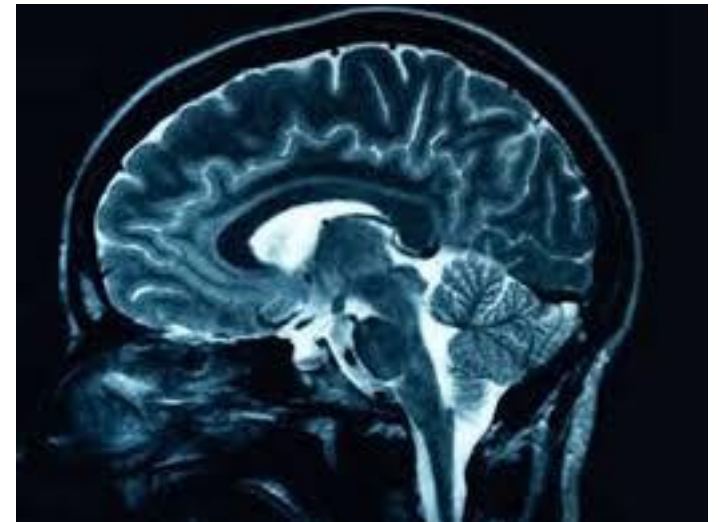
What's a data type where we can do this?



X-ray



Histopathology

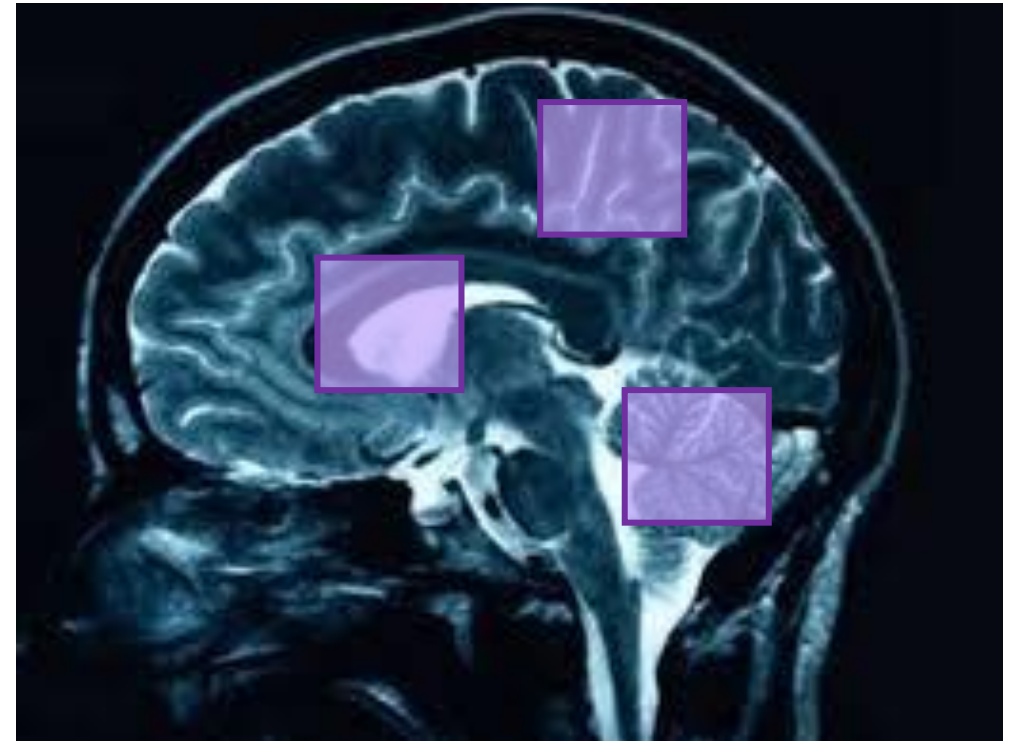


Magnetic Resonance Imaging (MRI)

Images!

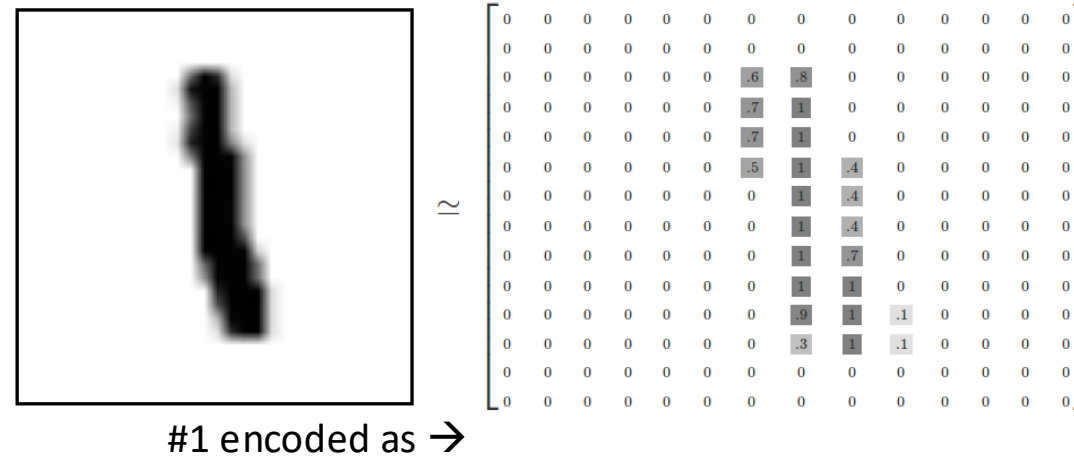
When partially connected networks are useful

- **Observation:** Nearby pixels are more likely to be related
- **Assumption:** It is okay to only connect the nearby pixels



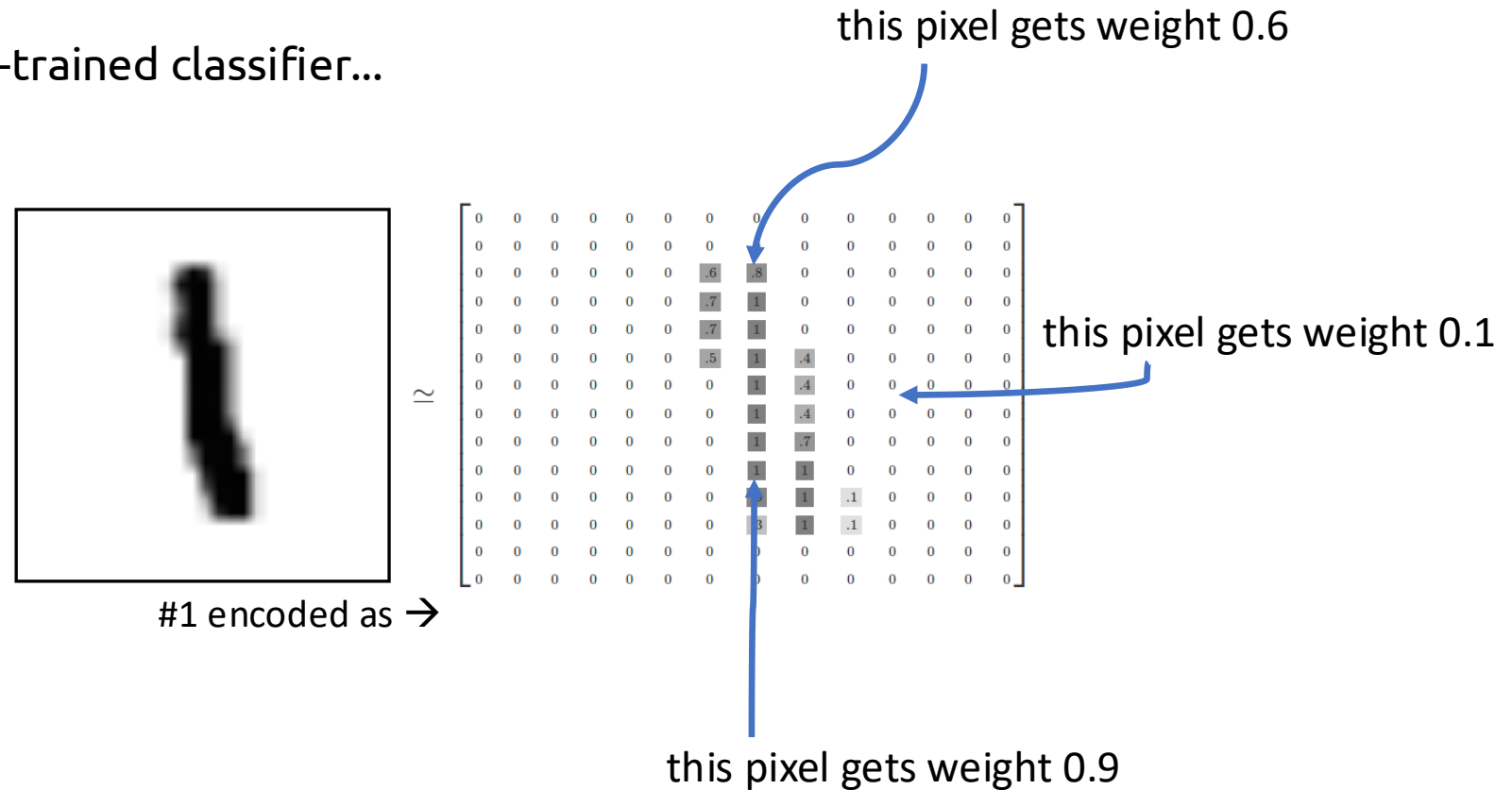
Limitations of Full Connections

Suppose we've got a well-trained classifier...



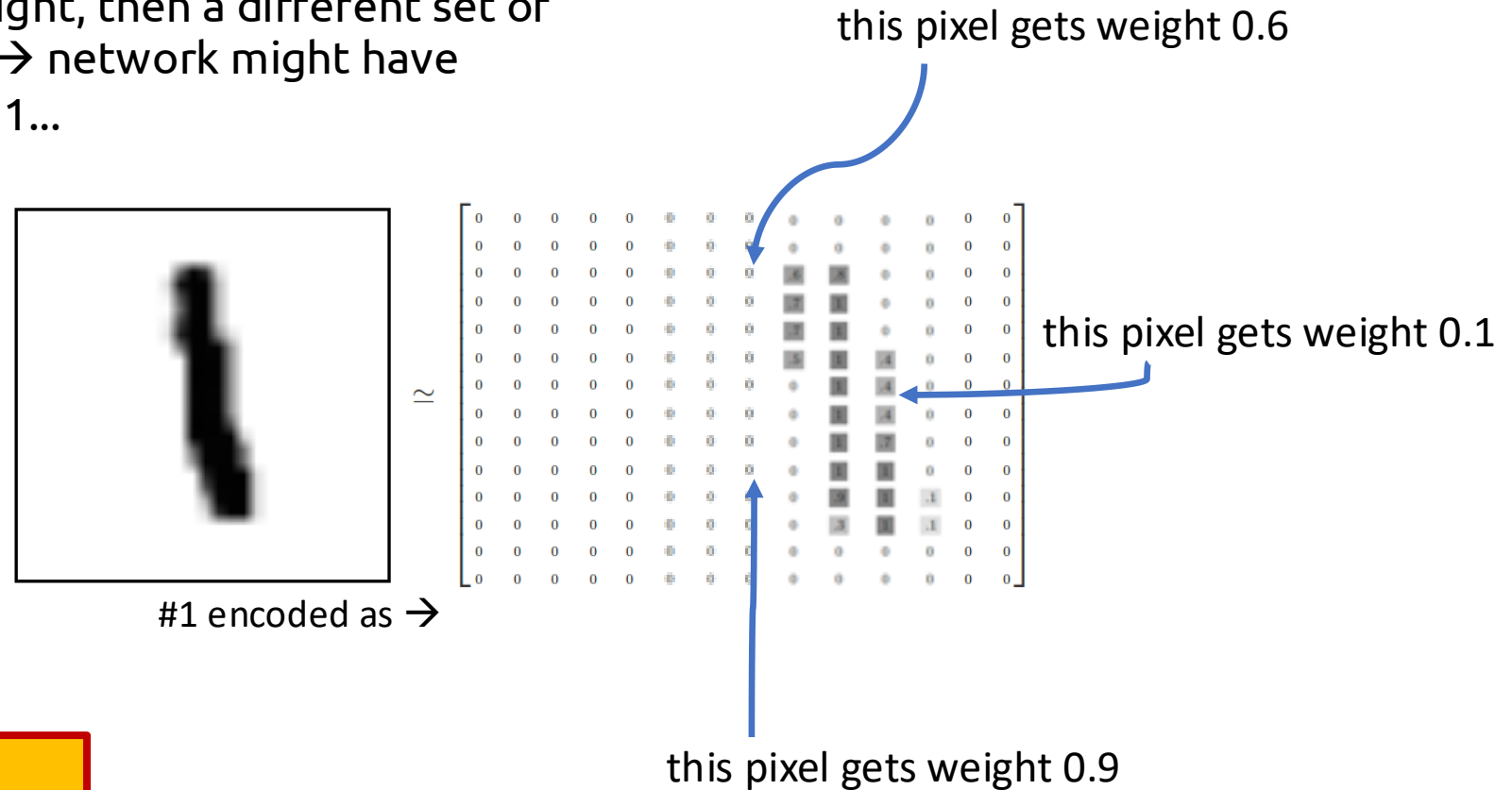
Limitations of Full Connections

Suppose we've got a well-trained classifier...



Limitations of Full Connections

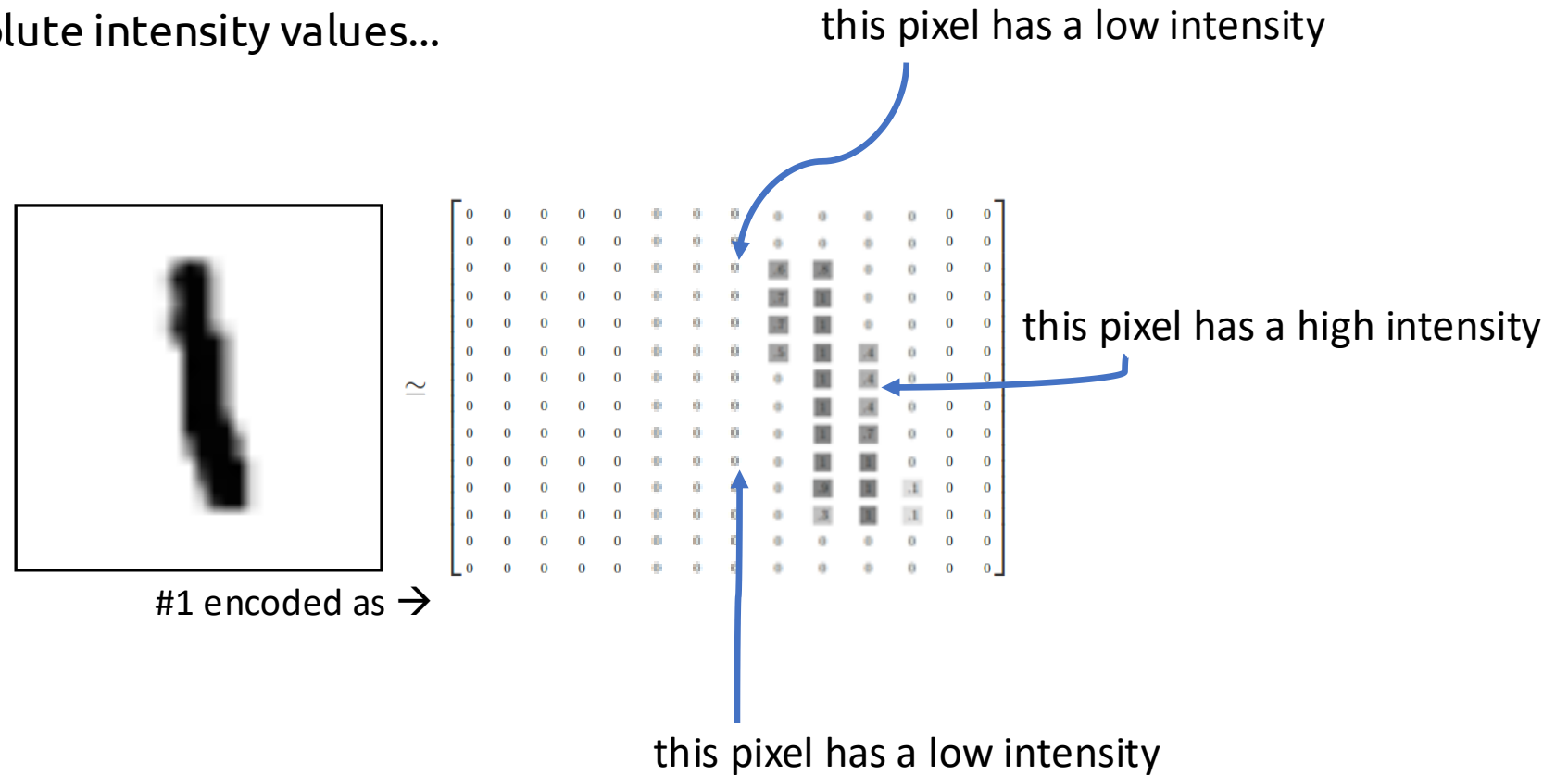
If we shift the digit to the right, then a different set of weights becomes relevant $\rightarrow$ network might have trouble classifying this as a 1...



Can you tell this is a 1?

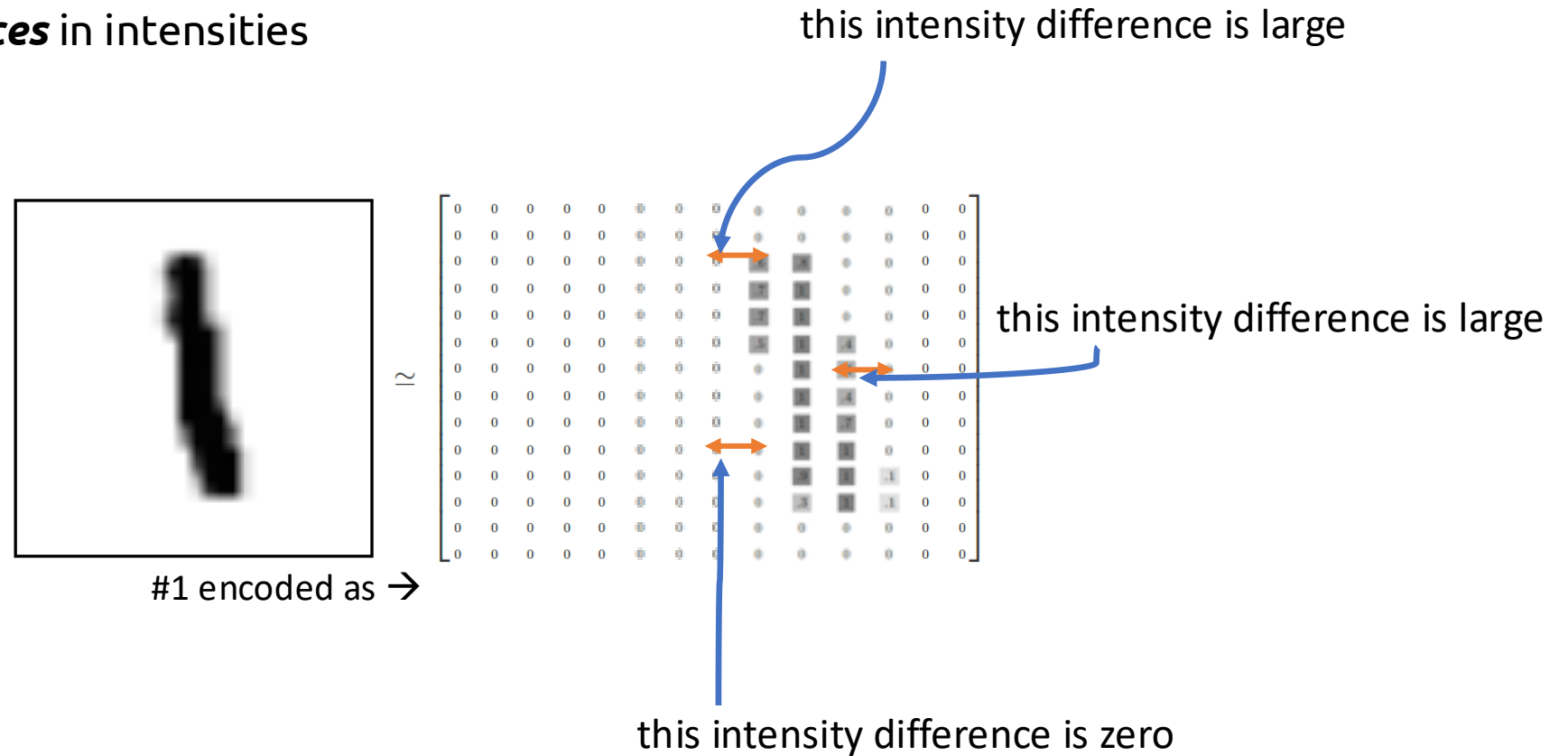
This would *not* be a problem for the human visual system

Our eyes don't look at absolute intensity values...



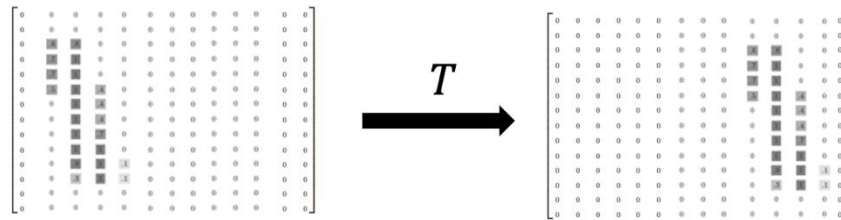
This would *not* be a problem for the human visual system

...but rather *local differences* in intensities



Translational Invariance

- To make a neural net f robust in this same way, it should ideally satisfy **translational invariance**: $f(T(x)) = f(x)$, where
 - x is the input image
 - T is a translation (i.e. a horizontal and/or vertical shift)

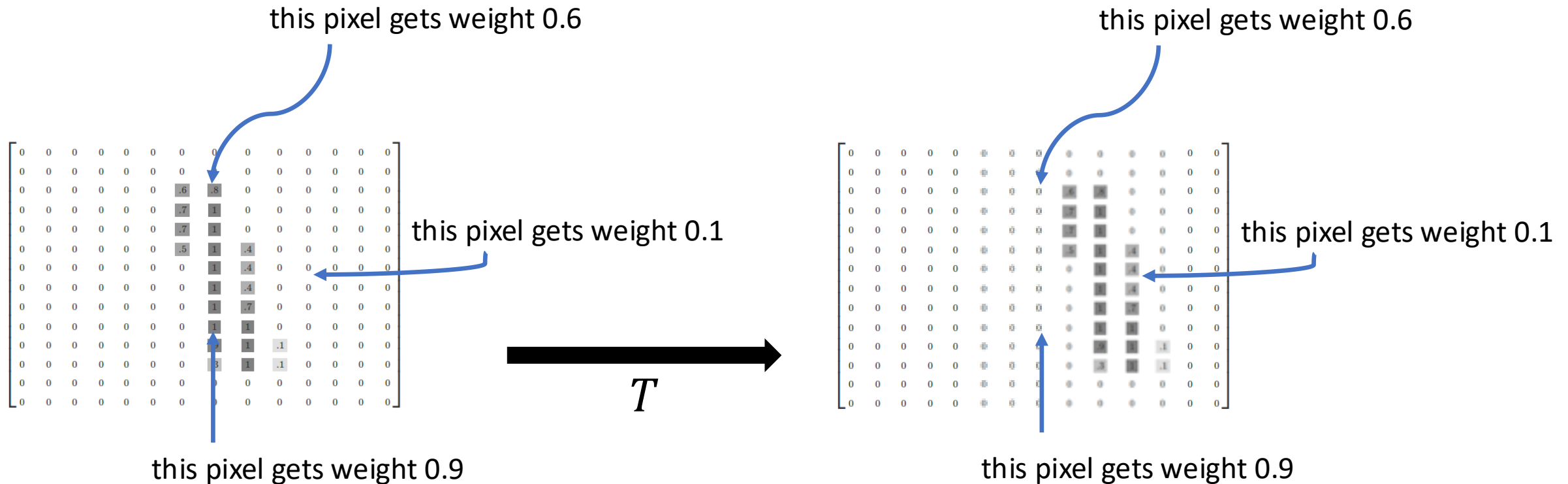


$$f\left(\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}\right) \stackrel{?}{=} f\left(\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}\right)$$

Fully Connected Nets are *not* Translationally Invariant

How to make the network translationally invariant?

Focus on local differences/patterns



Sum of these three: $0.6 \cdot 0.8 + 0.1 \cdot 0 + 0.9 \cdot 1 = 1.38$

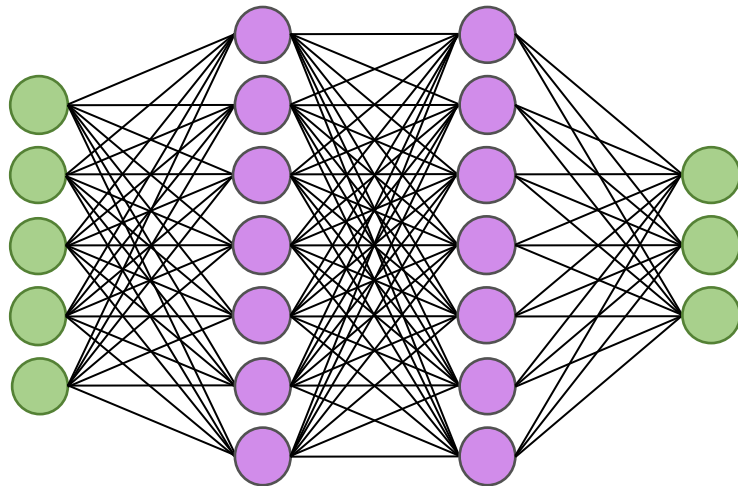
Sum of these three: $0.6 \cdot 0 + 0.1 \cdot 0.4 + 0.9 \cdot 0 = 0.4$

Focusing on local patterns = partial connections

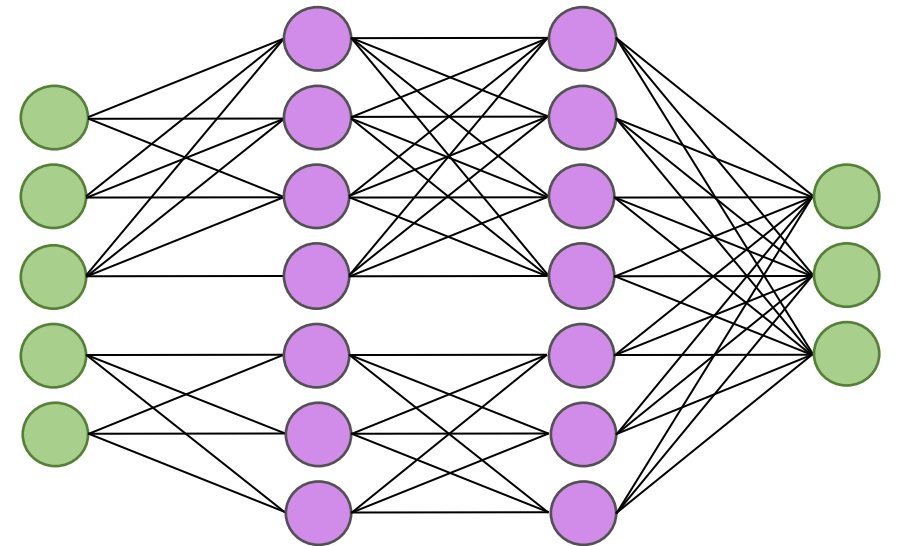
Any questions?



Fully Connected



Partially Connected



How do we do that?

The Main Building Block: Convolution

Convolution is an operation that takes two inputs:

(1) An image (2D – B/W)



(2) A filter (also called a kernel)

1	1	1
0	0	0
-1	-1	-1

2D array of numbers; could be any values

What Convolution Does (Visually)

image

2	0	1	3
7	1	1	0
0	2	5	0
0	5	1	4

filter/kernel

1	1	1
0	0	0
-1	-1	-1

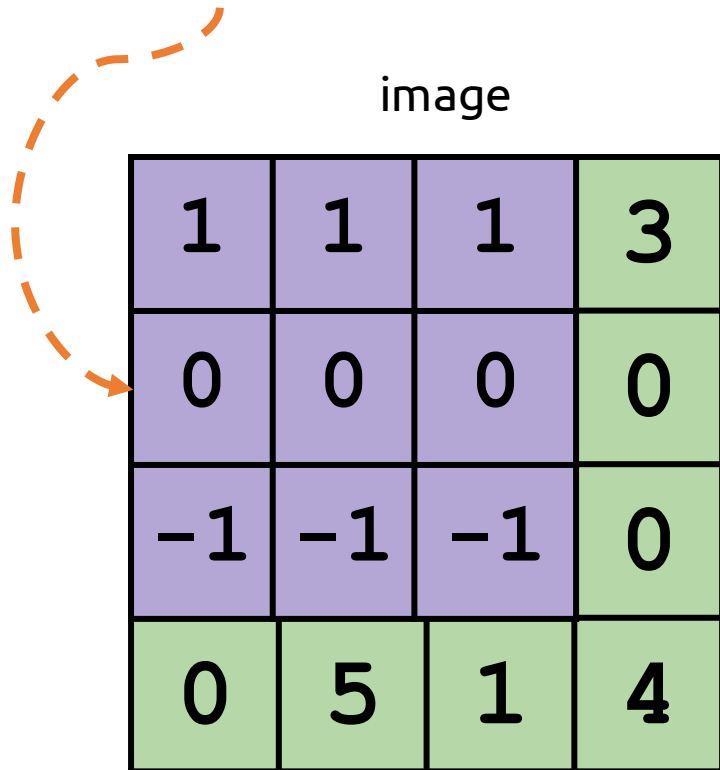


(We use this symbol for convolution)
(The verb form is “convolve”)

What Convolution Does (Visually)

Overlay the filter on the image

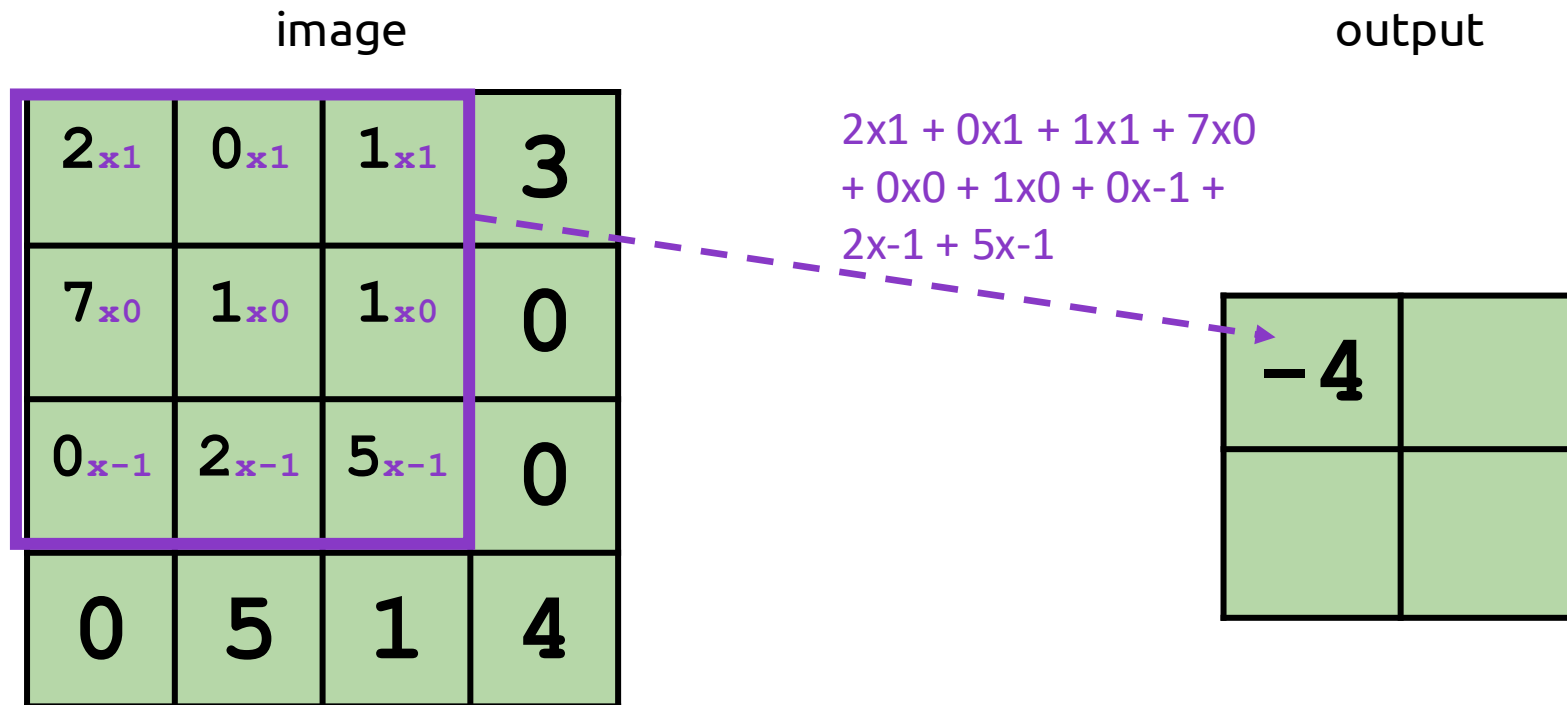
image



1	1	1	3
0	0	0	0
-1	-1	-1	0
0	5	1	4

What Convolution Does (Visually)

Sum up multiplied values to produce output value



What Convolution Does (Visually)

Move the filter over by one pixel

image

1	1	1	3
0	0	0	0
-1	-1	-1	0
0	5	1	4

output

-4	

What Convolution Does (Visually)

Move the filter over by one pixel

image

2	1	1	1
7	0	0	0
0	-1	-1	-1
0	5	1	4

output

-4	

What Convolution Does (Visually)

Repeat (multiply, sum up)

image

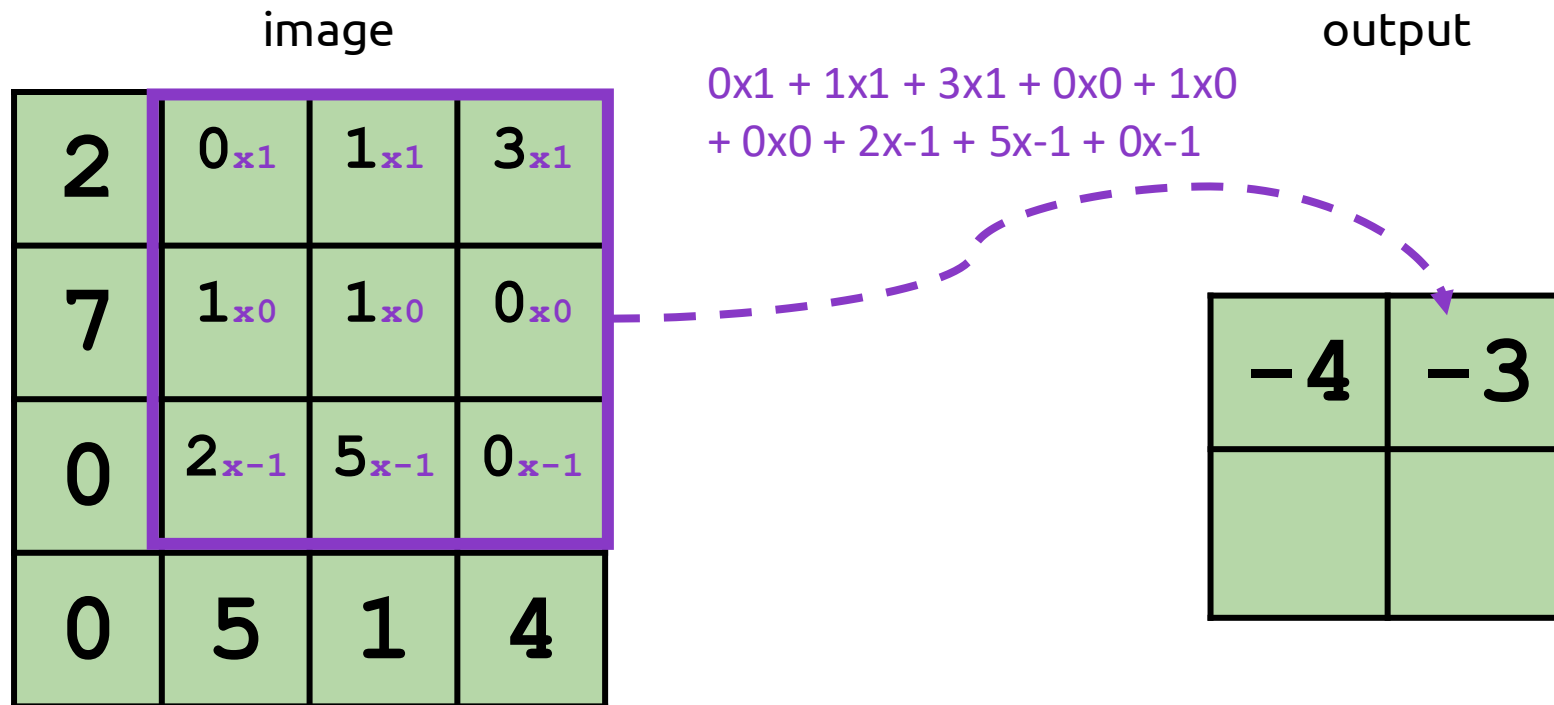
2	0 _{x1}	1 _{x1}	3 _{x1}
7	1 _{x0}	1 _{x0}	0 _{x0}
0	2 _{x-1}	5 _{x-1}	0 _{x-1}
0	5	1	4

output

-4	

What Convolution Does (Visually)

Repeat (multiply, sum up)



What Convolution Does (Visually)

Repeat...

image

2	0	1	3
7 _{x1}	1 _{x1}	1 _{x1}	0
0 _{x0}	2 _{x0}	5 _{x0}	0
0 _{x-1}	5 _{x-1}	1 _{x-1}	4

output

-4	-3
3	

$$7 \times 1 + 1 \times 1 + 1 \times 1 + 0 \times 0 + 2 \times 0 + 5 \times 0 + 0 \times -1 + 5 \times -1 + 1 \times -1$$



What Convolution Does (Visually)

Repeat...

image

2	0	1	3
7	1 _{x1}	1 _{x1}	0 _{x1}
0	2 _{x0}	5 _{x0}	0 _{x0}
0	5 _{x-1}	1 _{x-1}	4 _{x-1}

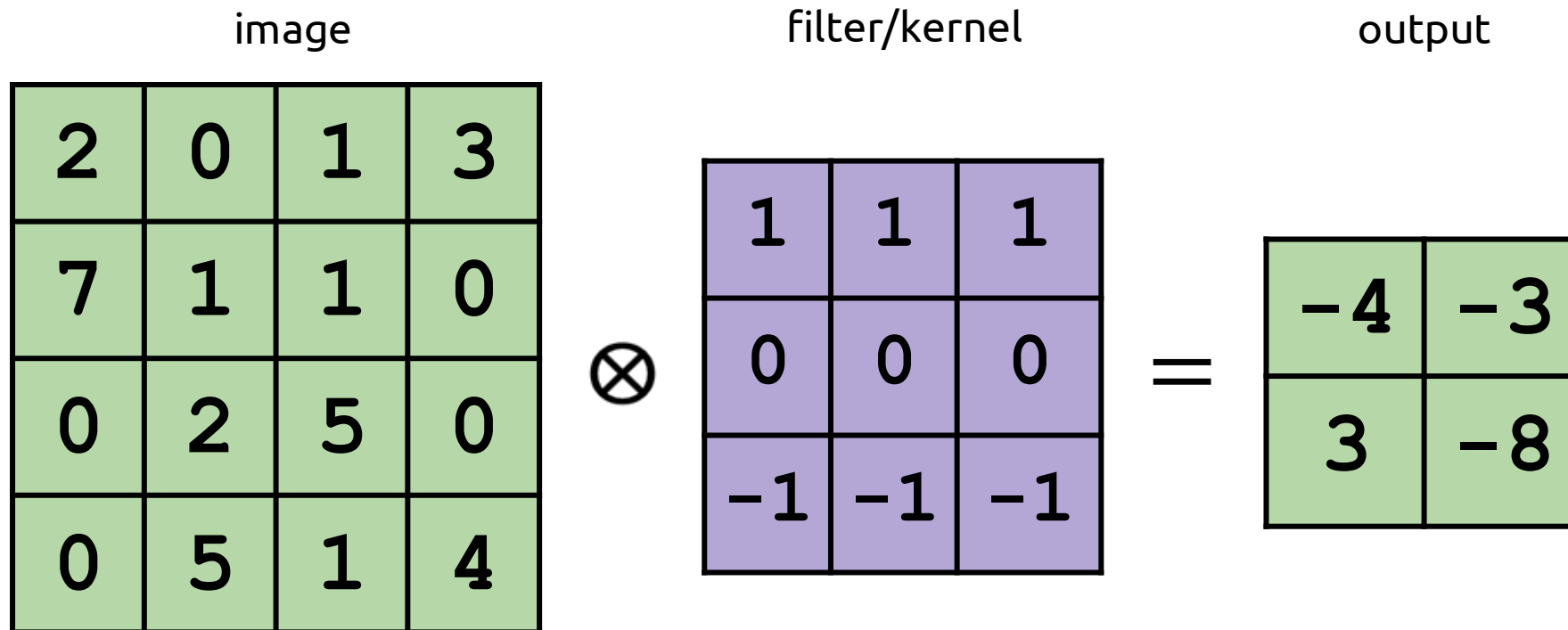
$$1 \times 1 + 1 \times 1 + 0 \times 1 + 2 \times 0 + 5 \times 0 \\ + 0 \times 0 + 5 \times -1 + 1 \times -1 + 4 \times -1$$

output

-4	-3
3	-8

What Convolution Does (Visually)

In summary:



Try it out yourself!

Convolve this
image

2	0	3	1
1	1	0	0
1	0	2	0
1	0	1	2



With this filter

1	0	-1
2	0	-2
1	0	-1

2	0	3	1
1	1	0	0
1	0	2	0
1	0	1	2

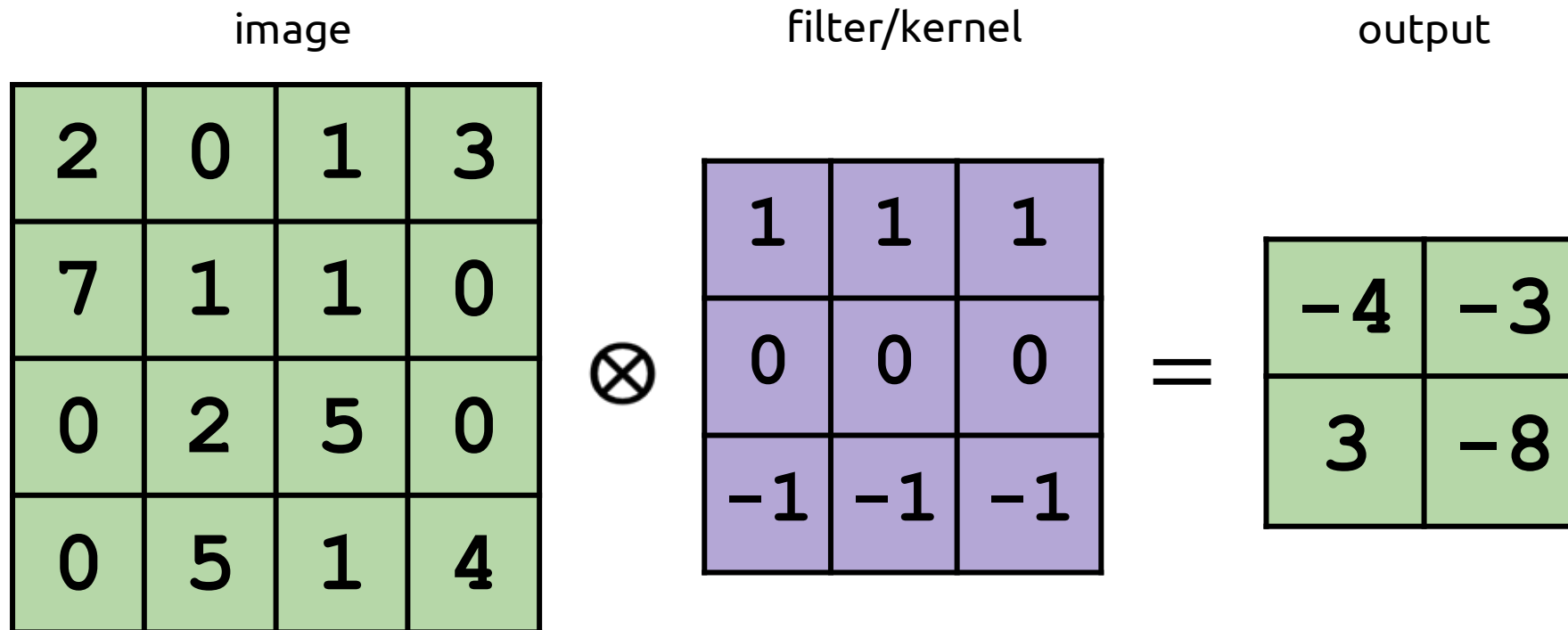
$\otimes$

1	0	-1
2	0	-2
1	0	-1

$=$

What Convolution Does (Visually)

In summary:



What Convolution Does (Mathematically)

$$V(x, y) = (I \otimes K)(x, y) = \sum_m \sum_n I(x + m, y + n) K(m, n)$$

The diagram illustrates the mathematical process of convolution. It features the equation $V(x, y) = (I \otimes K)(x, y) = \sum_m \sum_n I(x + m, y + n) K(m, n)$. Annotations with orange dotted arrows and lines explain each part:

- An arrow points from $V(x, y)$ to the text "The output at pixel (x, y) ".
- An arrow points from $(I \otimes K)$ to the text "'Image I convolved with kernel K '".
- An arrow points from the summation index m to the text "Sum over kernel columns".
- An arrow points from the summation index n to the text "Sum over kernel rows".
- A bracket connects the terms $I(x + m, y + n) K(m, n)$ to the text "Multiply kernel value with corresponding image pixel value".

What Convolution Does (Mathematically)

image

	x = 0	x = 1	x = 2	x = 3
y = 0	2	0	1	3
y = 1	7	1	1	0
y = 2	0	2	5	0
y = 3	0	5	1	4

filter/kernel

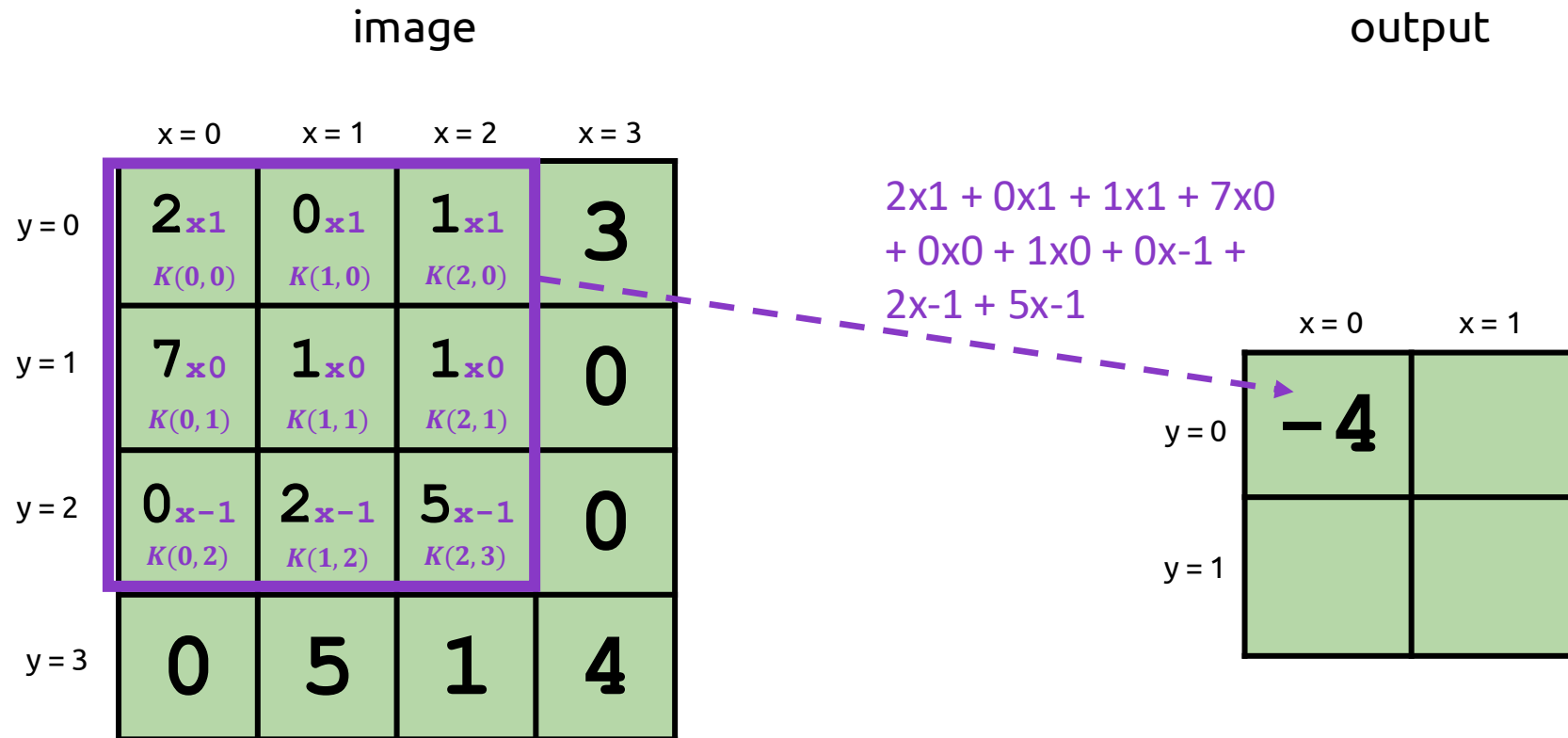
	m = 0	m = 1	m = 2
n = 0	1	1	1
n = 1	0	0	0
n = 2	-1	-1	-1



output

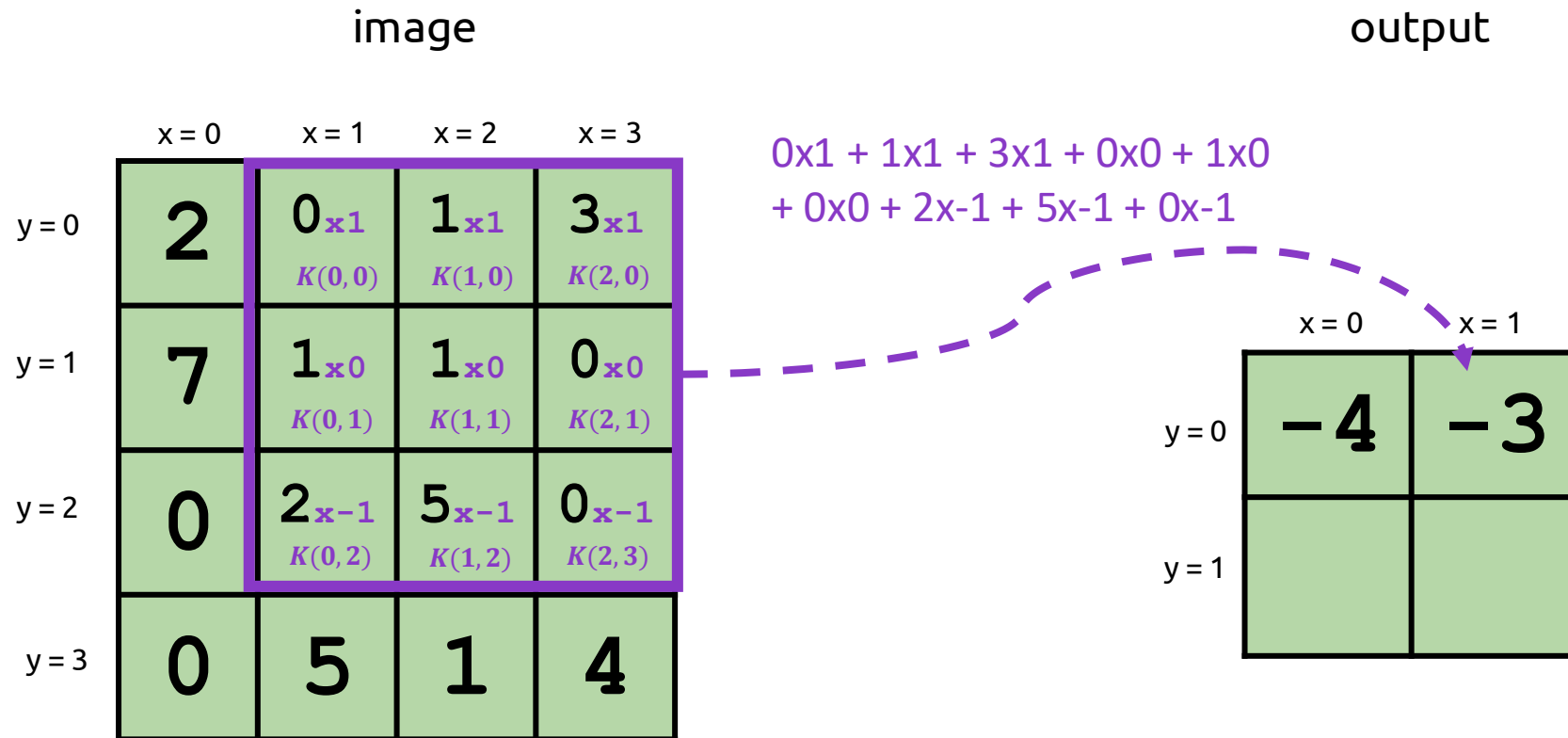
	x = 0	x = 1
y = 0	-4	-3
y = 1	3	-8

What Convolution Does (Mathematically)



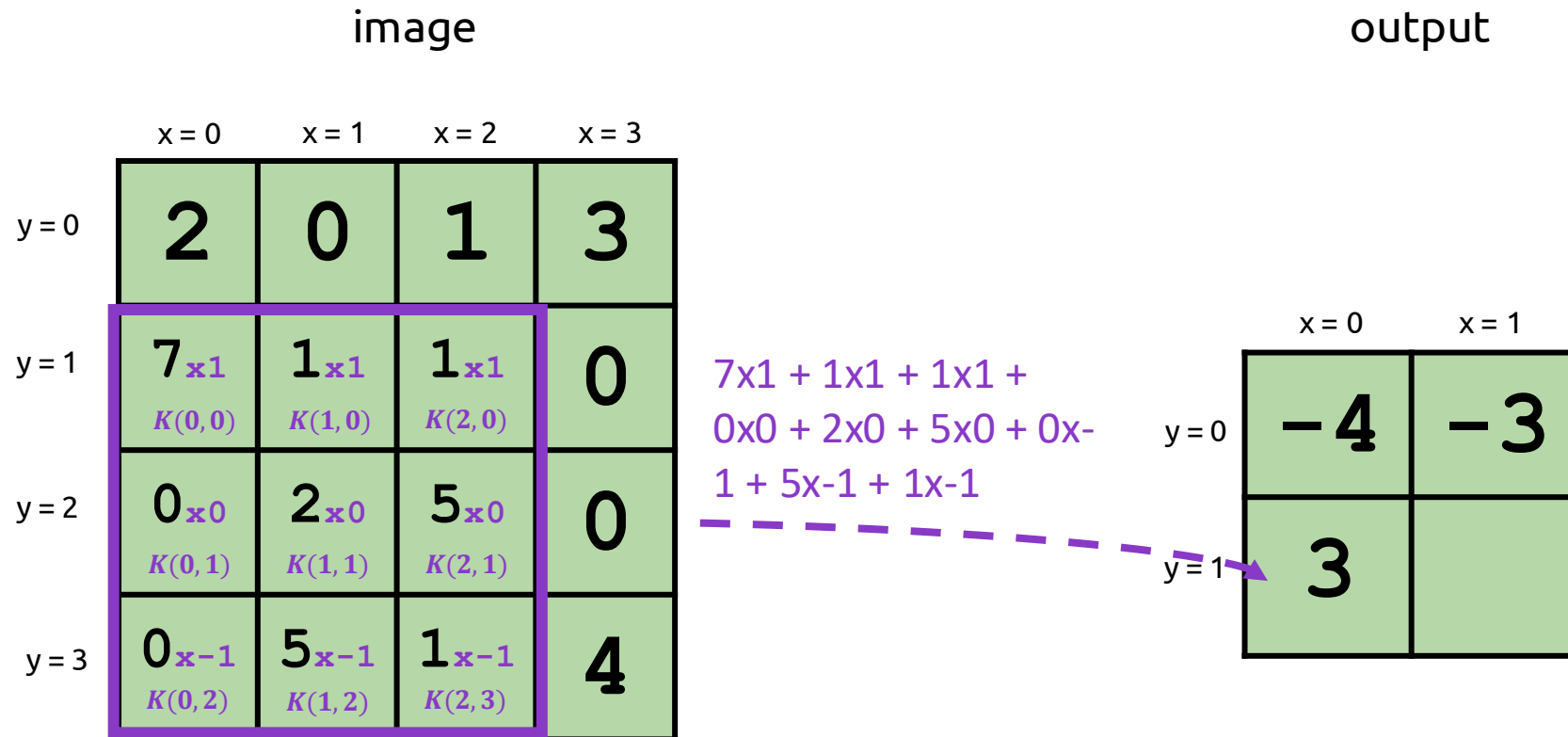
$$V(0,0) = (I \otimes K)(0,0) = \sum_{m=0}^2 \sum_{n=0}^2 I(0+m, 0+n) K(m,n)$$

What Convolution Does (Mathematically)



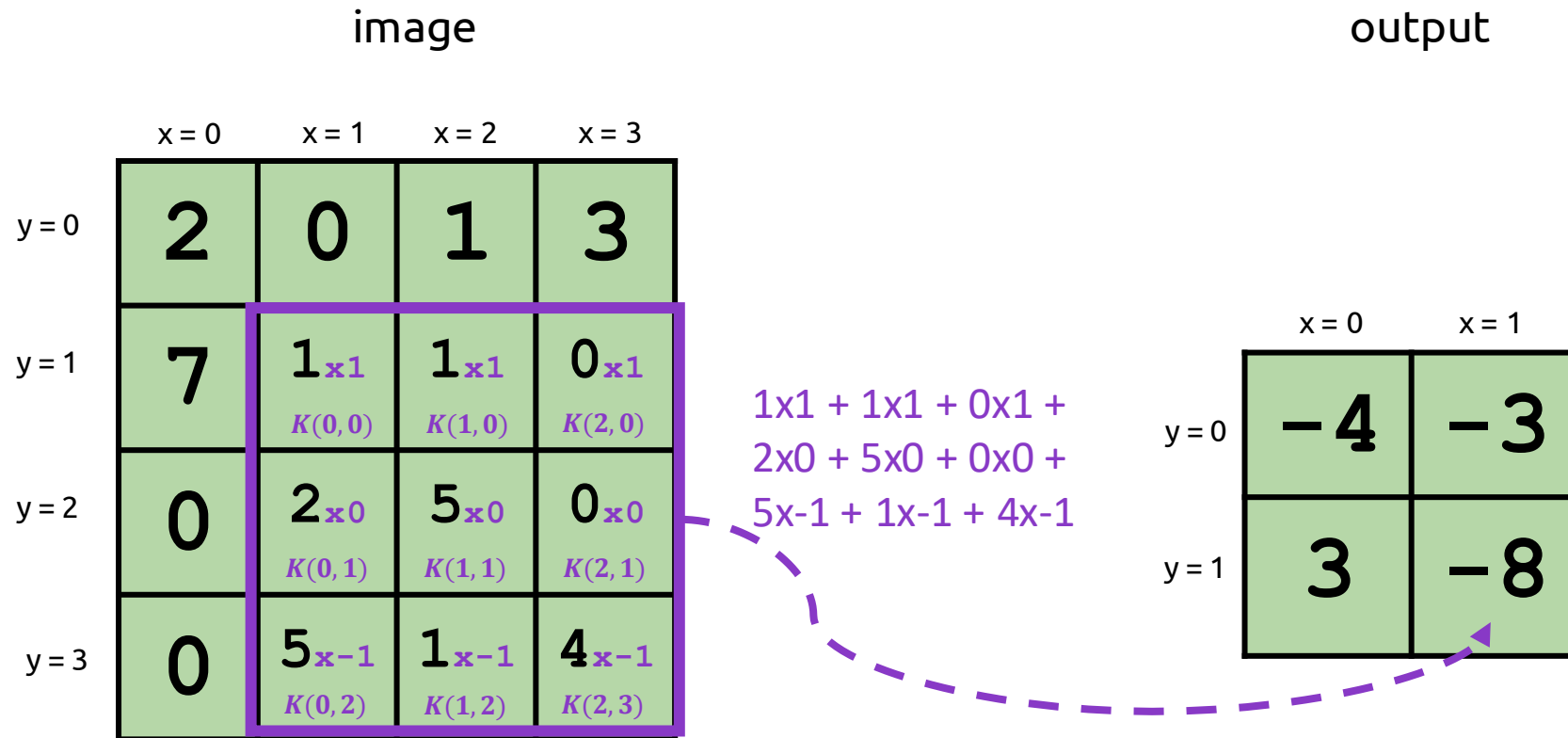
$$V(1, 0) = (I \otimes K)(1, 0) = \sum_{m=0}^2 \sum_{n=0}^2 I(1 + m, 0 + n) K(m, n)$$

What Convolution Does (Mathematically)



$$V(0, 1) = (I \otimes K)(0, 1) = \sum_{m=0}^2 \sum_{n=0}^2 I(0 + m, 1 + n) K(m, n)$$

What Convolution Does (Mathematically)



$$V(1, 1) = (I \otimes K)(1, 1) = \sum_{m=0}^2 \sum_{n=0}^2 I(1 + m, 1 + n) K(m, n)$$

What Convolution Does (In Code)

```
// Input: Image I, Kernel K, Output V, pixel index x,y
// Assumes K is 3x3
function apply_kernel(I, K, V, x, y)
    for m = 0 to 2:
        for n = 0 to 2:
            V(x,y) += K(m,n) * I(m+x, n+y)
```

Any questions?



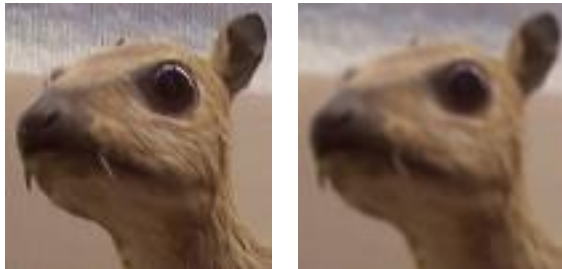
Equation: $V(x, y) = (I \otimes K)(x, y) = \sum_m \sum_n I(x + m, y + n) K(m, n)$

Different filters = different effects

<https://setosa.io/ev/image-kernels/>

Blur

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9



Edge Detection / Outline Kernel

0	-1	0
-1	5	-1
0	-1	0



Shift

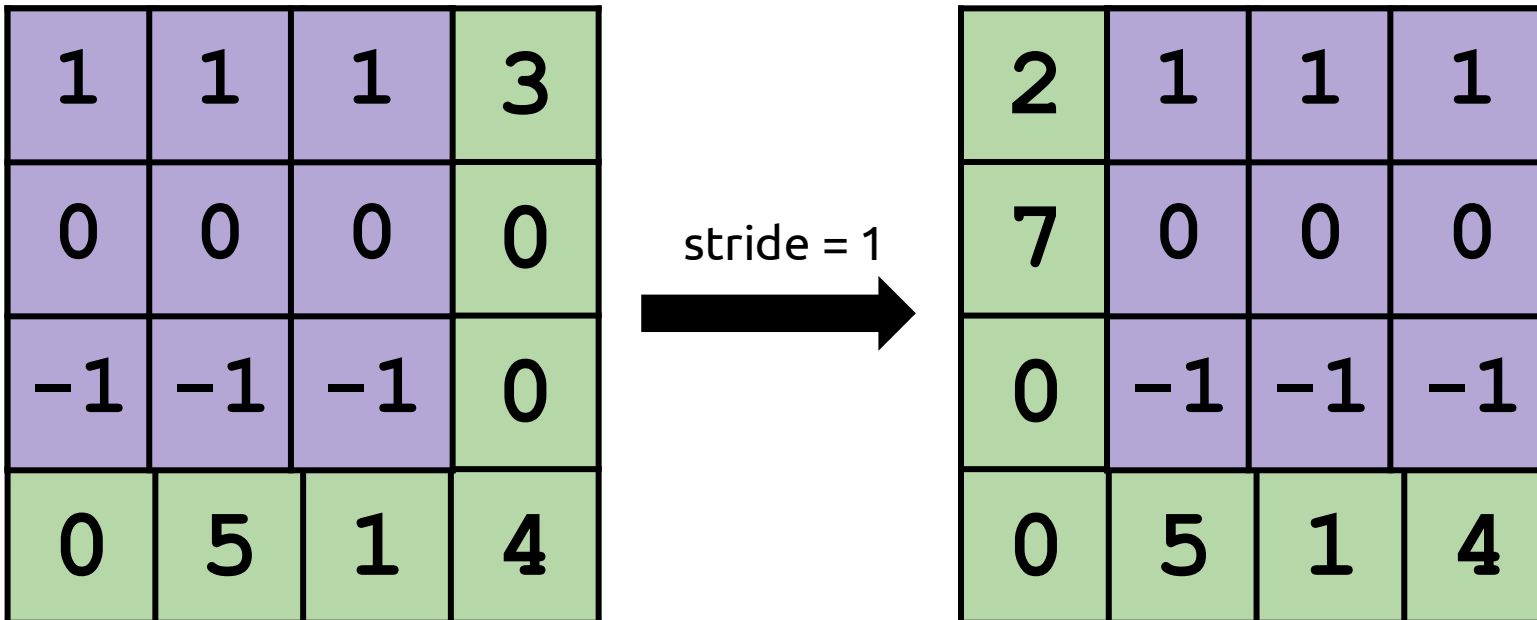
0	0	0
1	0	0
0	0	0



* exaggerated

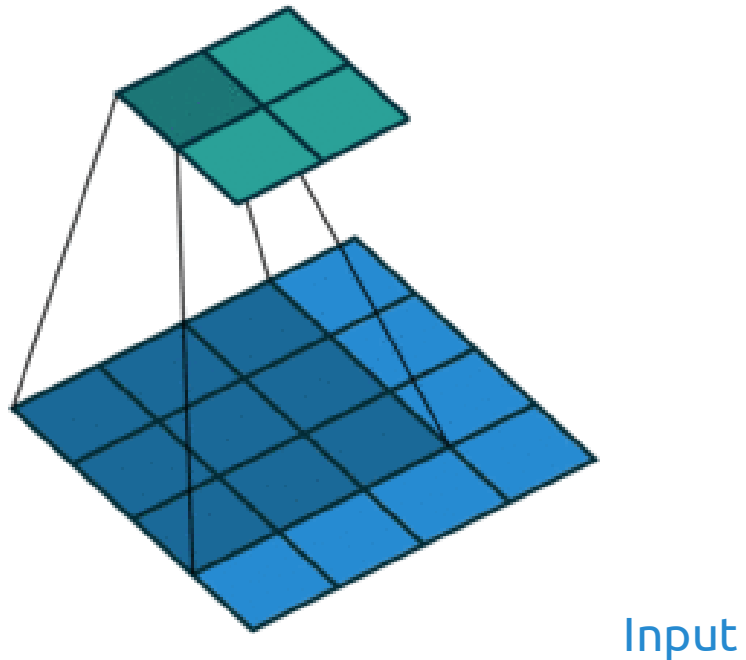
Stride

- We don't just have to slide the filter by one pixel every time
- The distance we slide a filter by is called ***stride***
 - All the examples we've seen thus far have been stride = 1

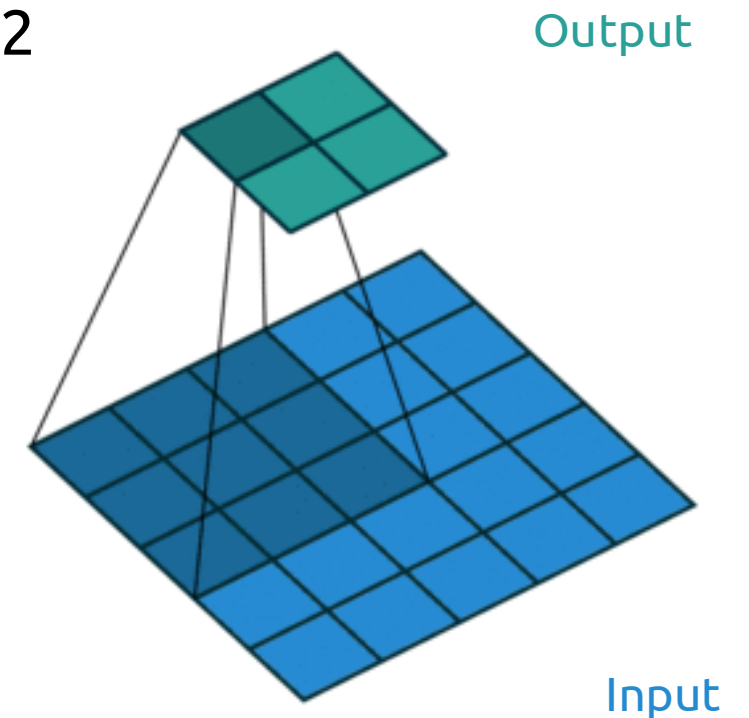


Stride in Action

Stride: 1

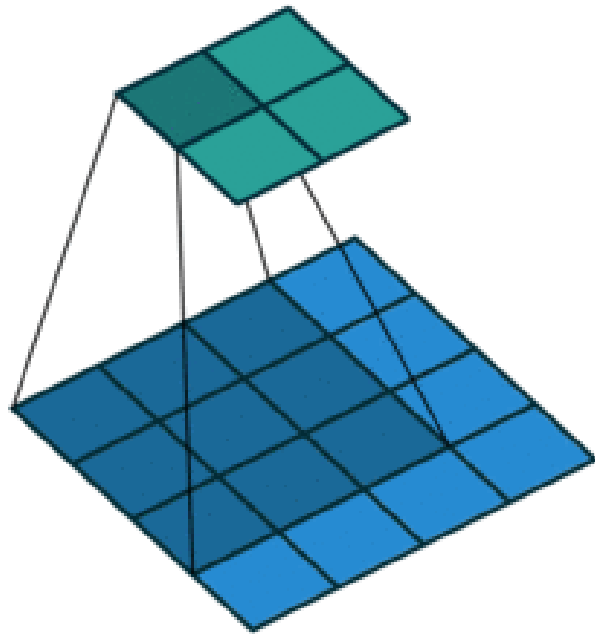


Stride: 2



Why would we want stride > 1?

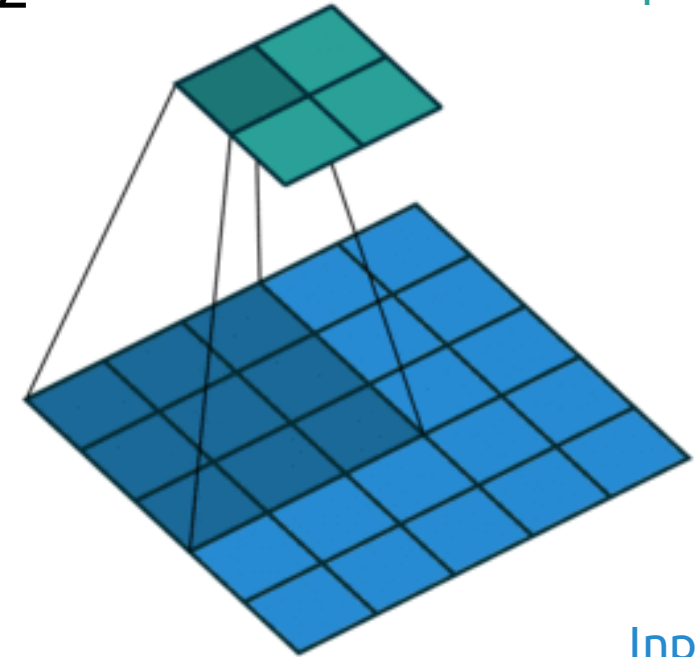
Stride: 1



Output

Input

Stride: 2



Output

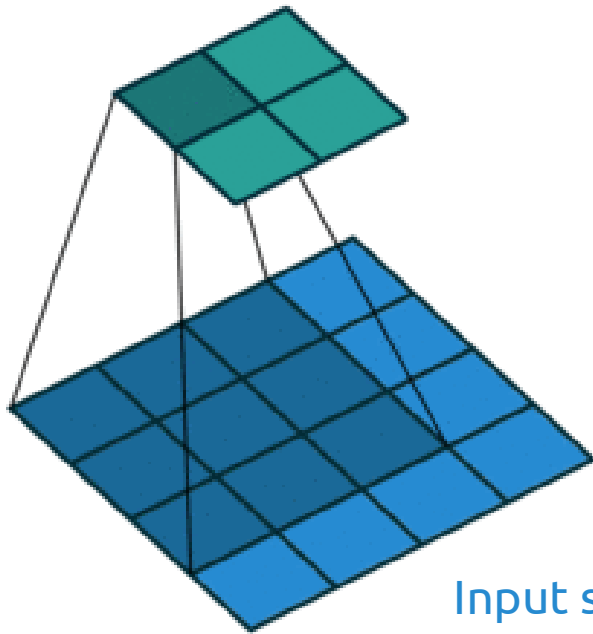
Input

Any connection
between input and
output size?

Why would we want stride > 1?

Stride: 1

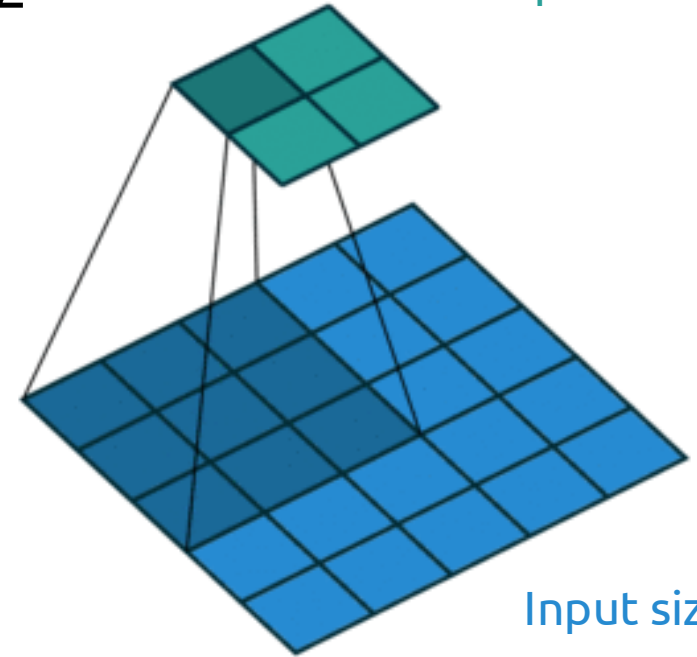
Output size: 2x2



Input size: 4x4

Stride: 2

Output size: 2x2



Input size: 5x5

Larger stride turns a bigger input into the same size output

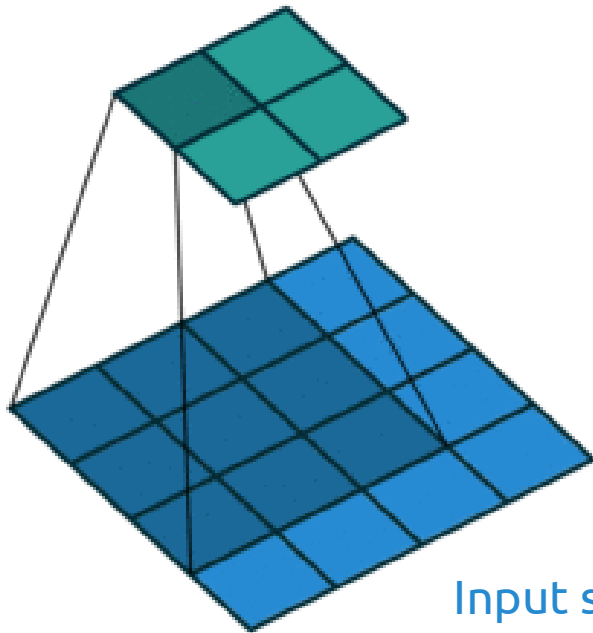
Any questions?



Why would we want stride > 1?

Stride: 1

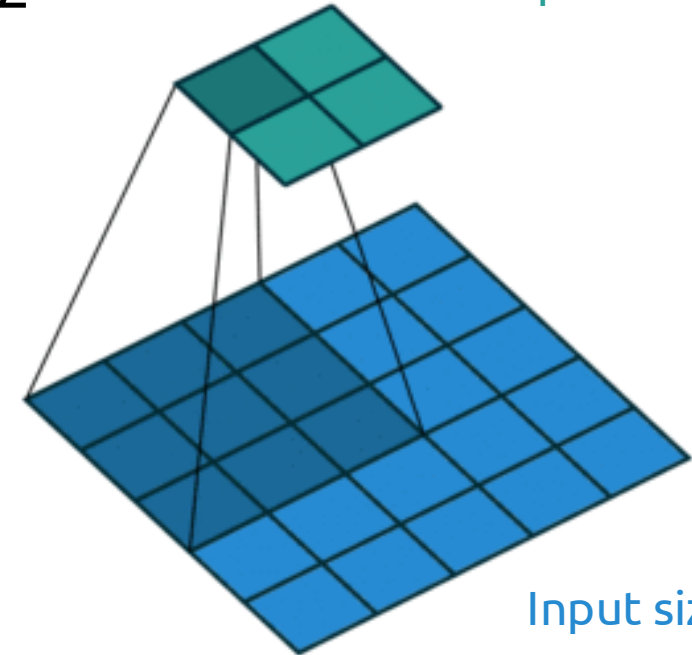
Output size: 2x2



Input size: 4x4

Stride: 2

Output size: 2x2



Input size: 5x5

Larger stride turns a bigger input into the same size output

Corollary: Larger stride turns the same size input into a *smaller* output

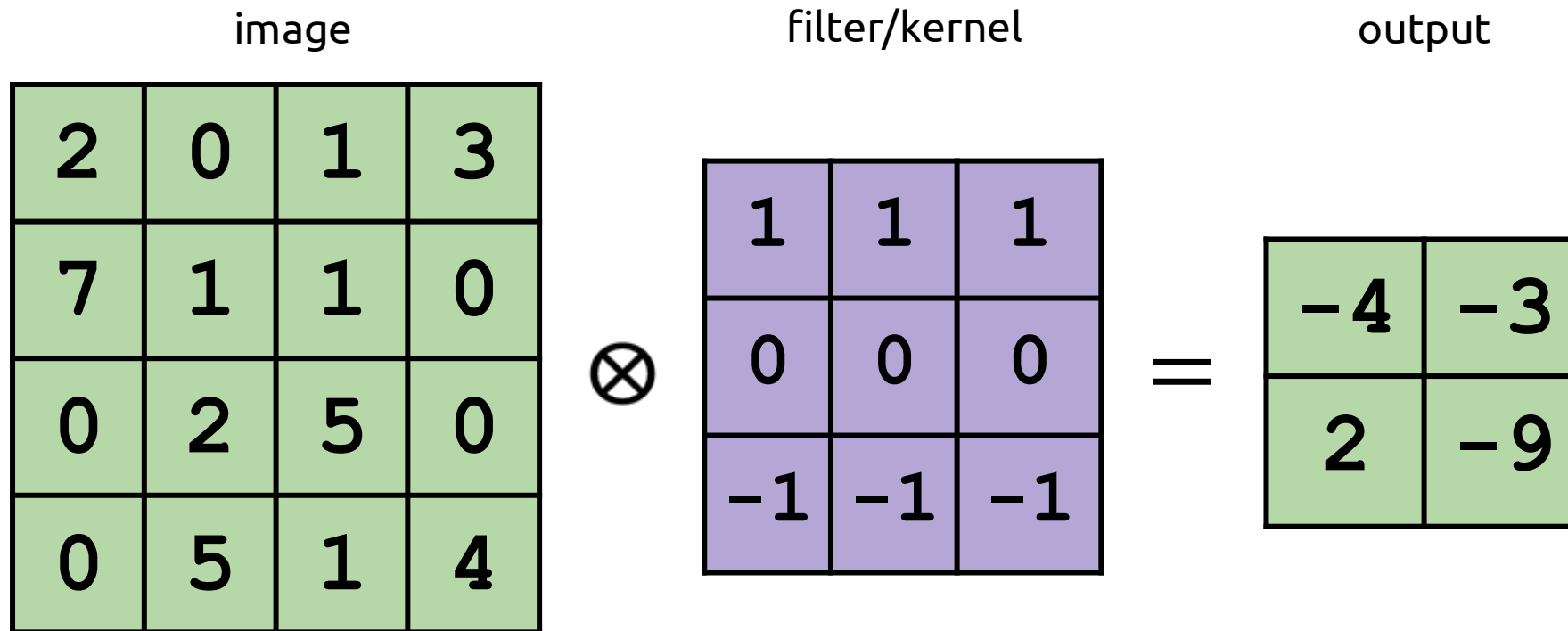
Use this to (controllably) decrease image resolution!

OK but...where's the *learning*?

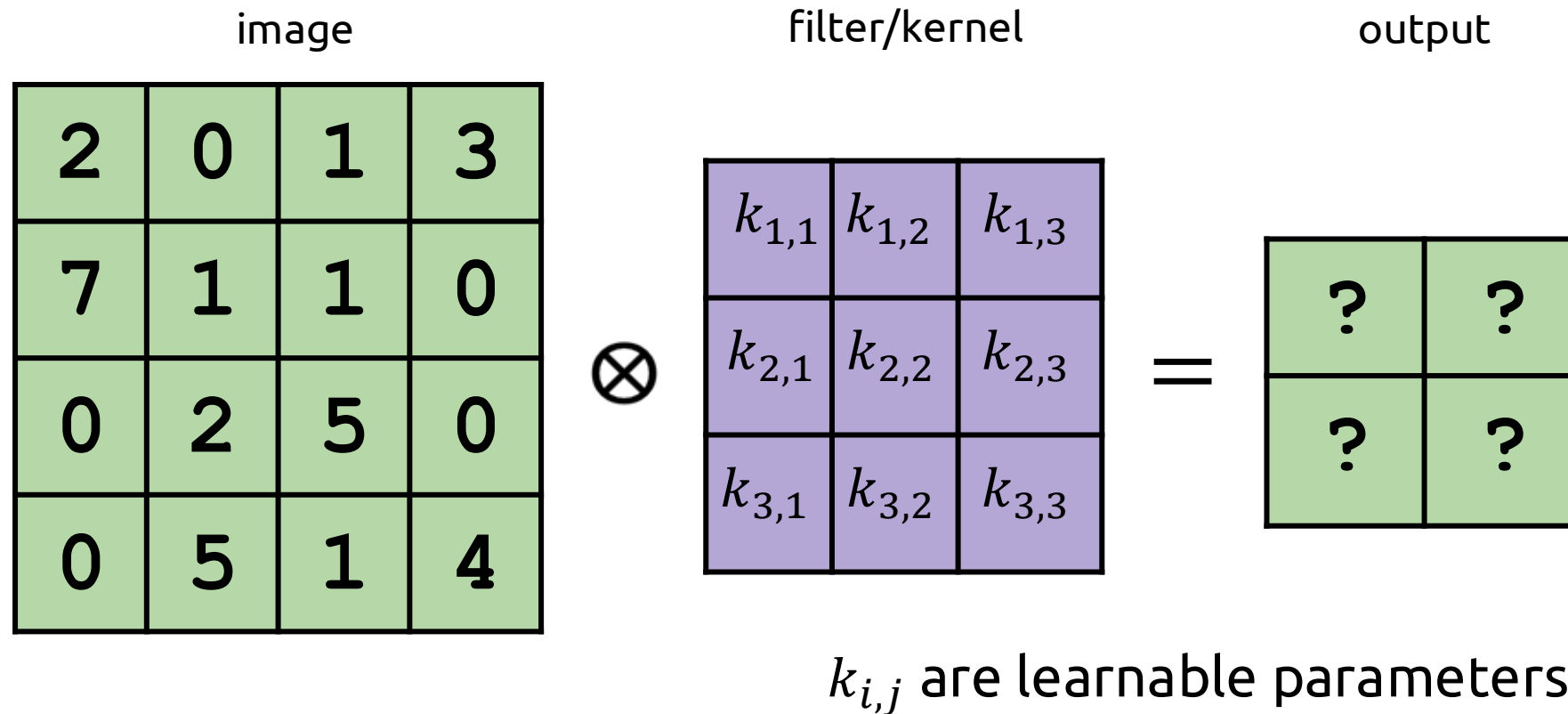
Can you guess what do we learn in
CNNs? (what are our parameters?)



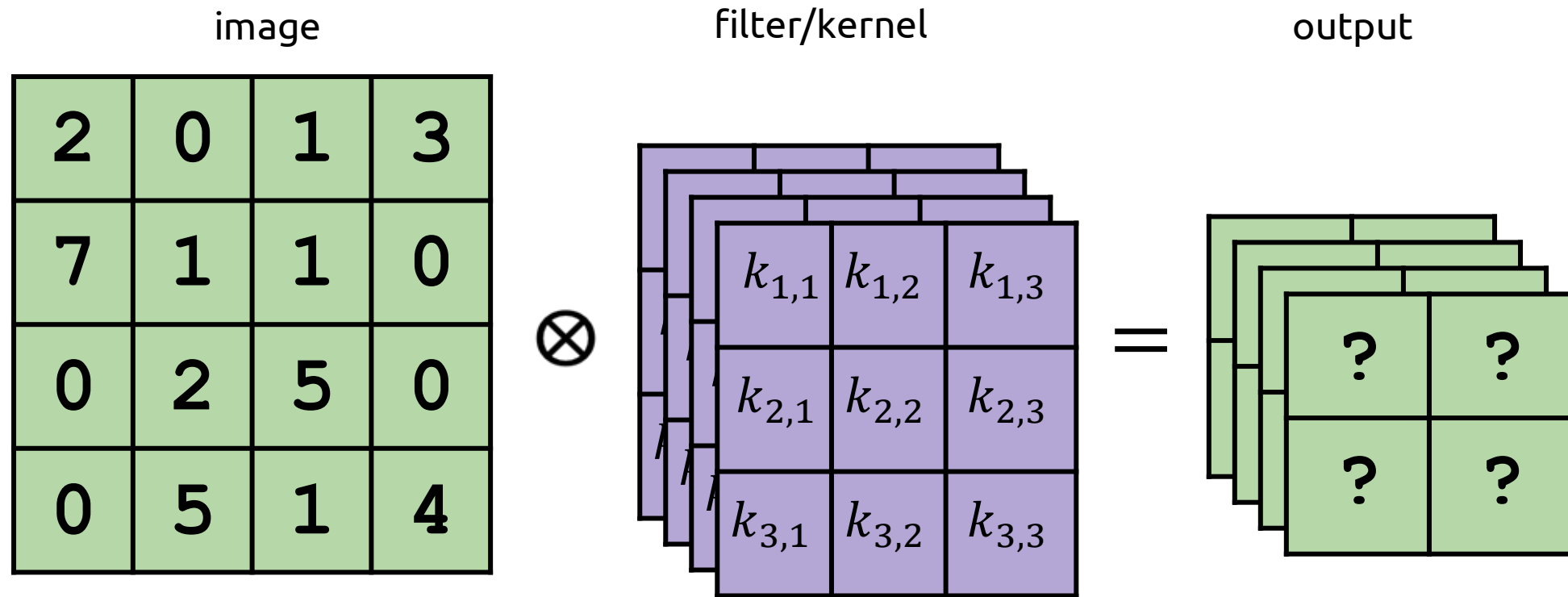
Key Idea 1: Filters are *Learnable*



Key Idea 1: Filters are *Learnable*

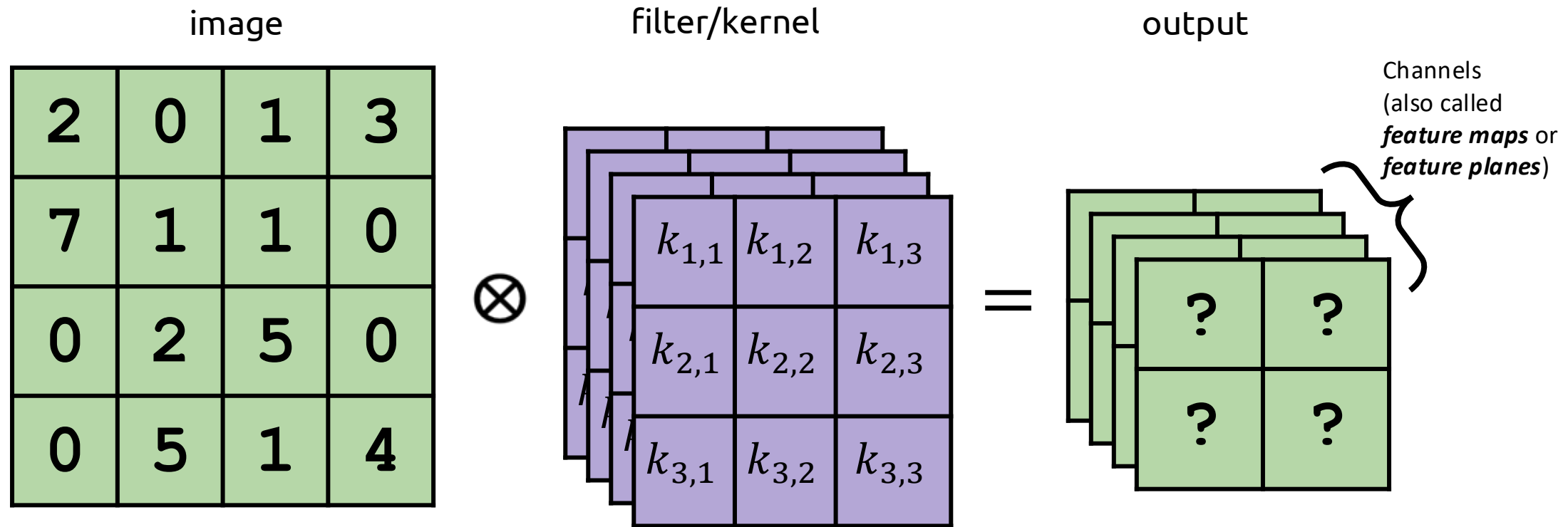


Key Idea 2: Learn *many* filters



This block of filters is called a ***filter bank***

Key Idea 2: Learn *many* filters



The output is now a multi-channel image

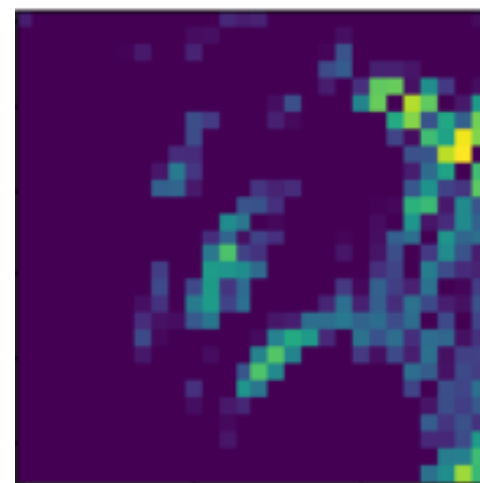


Key Idea 2: Learn *many* filters

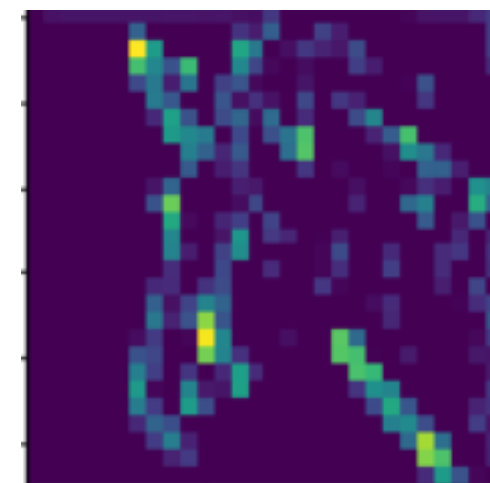
- Why are multiple filters a good idea?
 - Can learn to extract different *features* of the image



Input image



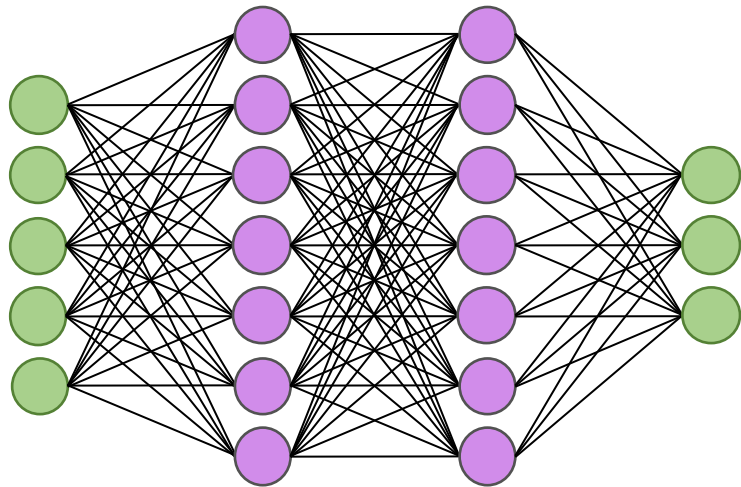
Output of filter 1



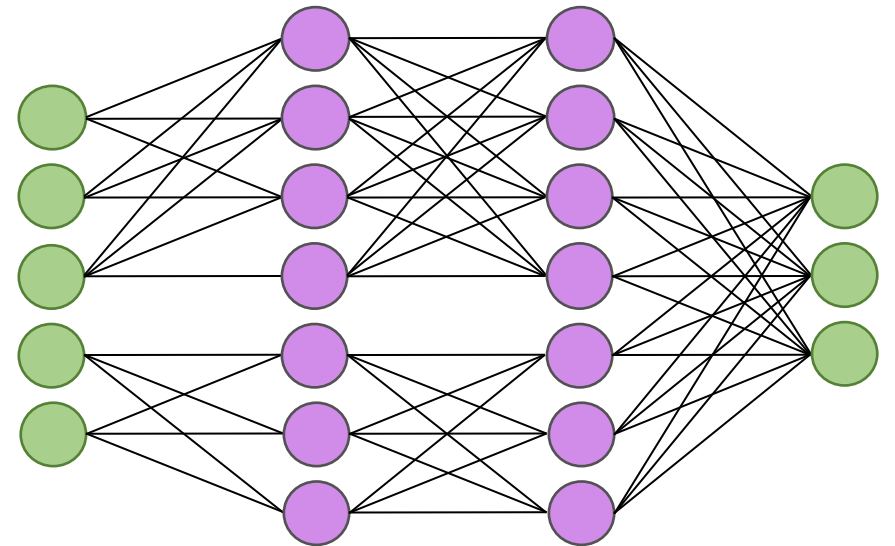
Output of filter 2

How is convolution “partially connected?”

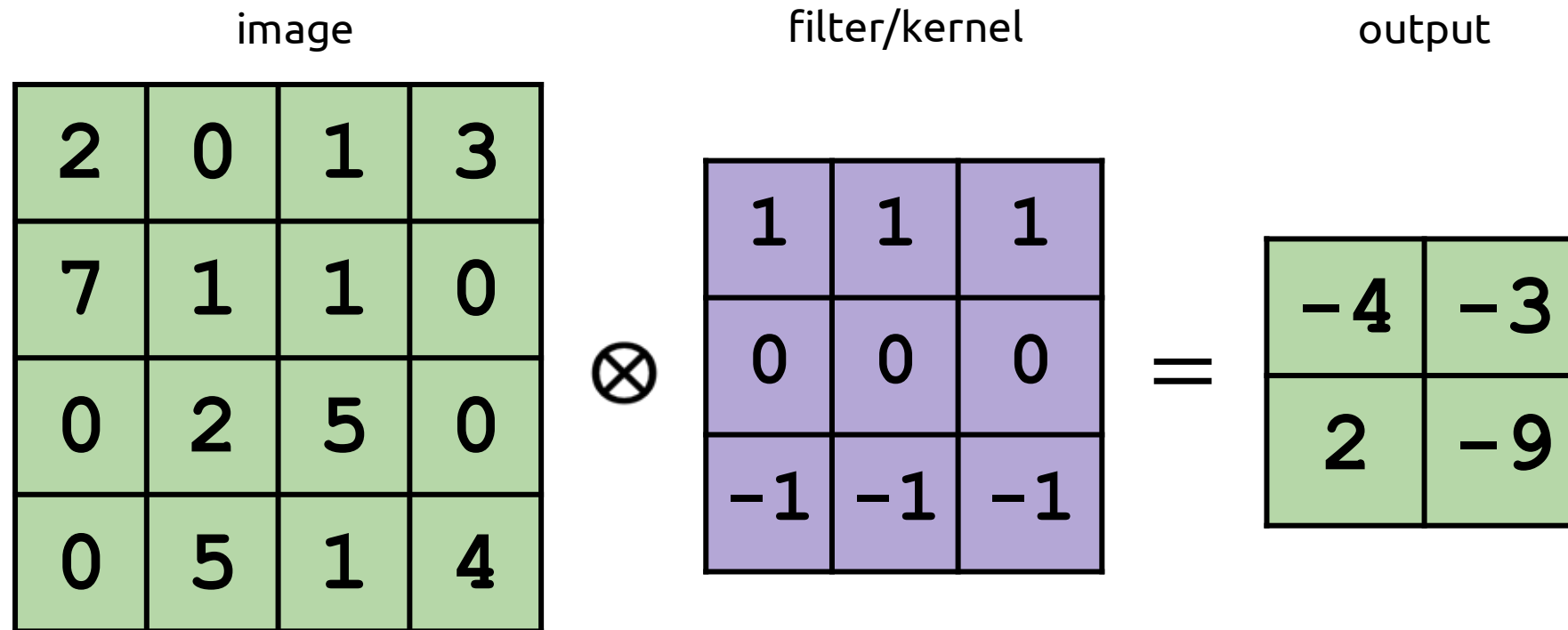
Fully Connected



Partially Connected



Only certain input pixels are “connected” to certain output pixels



Only certain input pixels are “connected” to certain output pixels

image

2	0	1	3
7	1	1	0
0	2	5	0
0	5	1	4

Colored dots in the input pixels represent which output pixels that input pixel contributes to

If this were fully connected, every input pixel would have all four output colors

output

-4	-3
2	-9

Any questions?



Recap – Learn about activations and Convolutional Neural Networks (CNNs)

(1) Adding layers to FCNNs

(2) Activation functions for non-linearity

(3) Class activity: Activation functions in action

(4) Convolution and CNNs

(5) Learning CNNs

Homework: [Chapter 5.1 and 5.2](#) and [Chapter 7.1 and 7.2](#) of the Dive into Deep Learning Book

Wrap up



What was the clearest point today?

What was the muddiest point today?

